# Left of the Loop

When AI writes the code, shared understanding becomes the scarce resource.

Simon Schrottner

REVIEW DRAFT — August 2026 · build unknown · not for distribution

# Contents

# A note on how this book was made

This book argues that AI can quietly erode the friction that produces shared understanding when we let it replace the conversations that build it. It was written with Claude. I'm aware of the irony.

The ideas, the stories, the framing: those are mine. But I struggle to structure an argument, and I miss my own contradictions. Claude closed that gap and made writing this possible at all. The moments that changed the book came from outside my own framing: readers pushing back on the argument, especially once I published it as a blog series before the book was finished. That pushback is the mechanism this book argues for, just in public rather than in a room.

Matthias, Johann, Bernhard, Carla, and Giulia read the first version of this book. It was the rough one, before any of this held together, and they told me what wasn't working. Thank you for getting through it.

I still don't know if that was enough. Judge for yourself. That's what the rest of this book is for.

# Introduction

This book is not what it looks like.

It looks like a book about AI: about agents, specs, workflows, and the changing shape of software development. Those things are in here. But they're not what the book is about.

What the book is about started with a university friend and 18 PlayStation Move controllers.

The two of us were building a demo for KubeCon. The talk was called "18 Bluetooth Controllers Walk Into a Bar," about observability and runtime configuration for JoustMania, an open-source party game where players jostle motion controllers until someone falls over. Complex execution: multiple Bluetooth adapters, battery-powered devices, sensors firing at 100Hz. When a player complains their controller "felt different," how does anyone debug it at 2 a.m. at a convention? Our goal was to figure out what was actually happening while it was still happening.

An interesting problem and two people who knew the domain and cared about the project.

As we had different schedules, we started hacking on our own. We both made progress and moved fast.

And we never planned it together.

The knowledge gap opened quietly. Decisions got made that the other person didn't know about. Assumptions turned out not to be shared. Work that should have built on itself didn't fit together. Not because either of us was wrong, but because we never built the shared picture before we started building the thing.

The fix wasn't better tools, a different framework, or a smarter approach to Bluetooth telemetry. What we'd skipped was the conversation.

This is happening to software teams everywhere right now, at a scale and speed that makes the gap harder to notice and more expensive to close.

AI gave every engineer a tool that makes starting to hack immediately feel productive. The agent is ready. The prompt is right there. Why stop to talk? Why slow down for the conversation? The output is coming, it looks right, the ticket will close. And somewhere in the gap between all that individual motion, the understanding stops being shared.

Engineers who go heads down with AI tools stop talking to each other. The output keeps coming, the confidence stays high, and the errors accumulate quietly, until a teammate catches them, until production does, or until nobody does.

AI has this problem for a structural reason. A model generates with the same confidence whether the output is right or wrong. The checks that catch it, the tests, the linters, the review, sit around the model, not inside it. Strip them away and nothing changes in the generation. No peer review. No colleague who reads it and says "that's not how it works."

AI hallucination and team misalignment aren't the same mechanism, but they rhyme: both produce confident output that nobody is checking against shared intent.

---

The industry's response to AI has been, mostly, to go faster.

Add the tool. Ship more. Reduce the headcount that feels redundant when the agent can implement. Optimize the individual. Measure the output. Skip the room.

That's a familiar move. The industry has cut the deliberate conversation before, rationally, because the process it had was tedious and wasn't bringing value, and learned, sometimes painfully, that the conversation was the work. How that happened, and how AI is offering the same deal at a larger scale, is the Phoenix's story to tell.

Extreme Programming (XP) understood what was at stake in the nineties.[1] Mob programming, pair programming, collective ownership: an entire practice built on the conviction that software development is a social activity. The understanding built through collaboration was the point.

The industry moved on. Faster frameworks. Individual metrics. Optimized handoffs. The team became a coordination cost to minimize.

The industry is rediscovering, under pressure, what XP already knew.

---

It begins with Sisyphus, and with me as the problem, pushing harder against a process that was already broken and shipping faster with AI without ever changing the terms of the curse.

It ends with the Phoenix, the choice to rise differently, a rediscovery of what was always true in a new form.

In between, it follows one thread, shared understanding, through its whole life. Where it belongs, what it is, and who builds it. What it's worth once implementation gets cheap. How a team creates it together, challenges it until it holds, and keeps it from dying when the people who held it move on. How you tell it's really there, what structure and culture keep it alive, and what it takes to scale it across an organization without thinning it to nothing.

The thesis: AI erodes the incidental friction that used to produce shared understanding as a byproduct of the work. That makes shared understanding the scarce resource, and constructive friction the mechanism that protects and tests it. Output got cheap faster than the ability to absorb it, and that is why this is happening now, at scale. The organizations that preserve the constructive kind will outperform the ones that optimize it away.

This has happened before, and the shape repeats. Assembly gave way to compilers and the hard part moved from getting the instructions right to designing the program. Frameworks absorbed the plumbing and the hard part moved to modeling the domain. Cloud absorbed the servers, open source absorbed whole categories

[1] Kent Beck, *Extreme Programming Explained: Embrace Change* (Addison-Wesley, 1999).

of code, and each time the scarce thing moved further from the machine and closer to intent. Agents are the largest instance of that move, not a departure from it. Which is why the argument here should outlast the tooling that prompted it: what gets cheap changes, and the direction the constraint travels doesn't.[2]

This is a practitioner's book. It doesn't claim to be the first to notice that teams matter or that shared understanding is valuable. Those ideas have lineage: Peter Naur's theory-building view,[3] XP, domain-driven design,[4] collective code ownership, and an academic literature of their own.[5] What's new is the gap: every current tool and framework for AI-assisted development solves alignment between one person and one agent. None of them address what happens when the spec needs to emerge from a room of people who don't yet share the same understanding. The spec: what to build and why, held in common. Not a document.[6] This book is the argument for that layer, the human and social conditions that make any spec-first approach work.

The conversation that feels slower than prompting alone
is doing something the prompt cannot do.

---

The argument in this book didn't emerge in isolation. While writing it, I kept finding people who'd arrived at the same conclusions on

[2] Fred Brooks made the general case in "No Silver Bullet: Essence and Accident in Software Engineering" (1986; reprinted in *The Mythical Man-Month*, anniversary edition, Addison-Wesley, 1995). The accidental complexity of software — the part tooling can remove — keeps falling, while the essential complexity of specifying and designing the conceptual construct does not. This book is an argument about what happens to the essential part when a very large amount of the accidental part disappears at once.

[3] Peter Naur, "Programming as Theory Building" (1985; reprinted in *Computing: A Human Activity*, ACM Press/Addison-Wesley, 1992). Naur argued that a program is the theory its builders hold, that the theory cannot be fully captured in documentation, and that a program whose builders have dispersed is effectively dead.

[4] Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Addison-Wesley, 2003).

[5] Martin Glinz and Samuel Fricker, "On Shared Understanding in Software Engineering: An Essay," *Computer Science — Research and Development* 30, nos. 3–4 (2015): 363–376.

[6] Throughout this book, *spec* means this shared understanding. Where the formal sense is meant — a contract, like the OpenFeature specification — the full word is used.

their own. The pressure toward this model is real enough that people keep hitting it without coordinating.

I started writing this book alone, with no one to tell me whether the idea was right. An argument for teams, begun without one, which is why the absence shapes its early chapters. The readers came later, once there was a draft to show.

There is a second absence, and it runs the length of the book. The failures in it happened, and I was there for all of them. The practice I propose in answer has no comparable record, because nobody has been running these long enough to have one, and the session in Appendix C is one I reconstructed rather than one I sat in. The diagnosis is evidence. The prescription is a bet, and I would rather you took it knowing that.

If you feel like something is wrong but can't point to it, if the speed feels good and the output looks right but something is missing, this book was written for you.

It's written for engineers, but also for the people in the room with them. Product owners trying to understand why AI adoption hasn't made delivery more predictable. Team leads watching their teams drift apart. Stakeholders wondering why the spec keeps getting misunderstood. If you care about how software gets built together, you're in the right place.

You're not behind. You're seeing what the speed costs.

We started hacking. We forgot to talk to each other. We forgot to build the picture together.

This is the conversation we should have had first.

# The Curse of Sisyphus

*Sisyphus was condemned by the gods to roll a boulder up a hill for eternity, only to watch it roll back down each time he neared the top. Not punishment for failure, but punishment for believing he could outsmart the system.*

*If implementation is the cheap part, where does the hard part go?*

I was the problem.

That took me a while to see. I was shipping fast, quality was good, tests were there. Every review request I opened was a genuine improvement to the codebase. And I had multiple open at any given time, because I was filling the review wait with more implementation. While someone in Canada was sleeping, I was three tasks deep: a new review request open, another under review, a third being planned.

The team was distributed, six hours between Europe and Canada, but that wasn't the issue. Distributed teams work. The issue was volume. AI let me produce faster than the team could absorb. Review requests piled up. I started stacking them to manage the backlog, which created more noise. The system was designed to produce exactly that outcome: a process that couldn't keep pace with the output.

I felt the burden. No one said anything directly, but you feel it in the silence. Review requests sitting. Comments sparse. The sense of generating more than the team can hold.

So I did what felt productive. I opened another review request.

We had something close to the right process. A product manager and the more senior engineers would talk through what needed building. Decisions got made and intent got written into Confluence. Then it became tickets, and the team picked and went. That sounds reasonable, but two things were broken simultaneously.

The scoping documents were written from a product manager's perspective: where we want to go, what the final state should look like, the vision six months out. Useful for a roadmap. Not useful when an engineer is trying to decide how a function should behave today. And when the docs did go technical, they'd sometimes arrive with a solution already decided by the seniors and the product manager, handed down to whoever picked the ticket. Or they'd leave the technical decisions to the engineer, mid-implementation, without the context the seniors had built up in their informal conversations.

We had also stopped doing planning sessions. I'm not sure exactly when. It wasn't a decision. It just faded. The backlog existed, the docs existed, the tickets existed. Planning felt redundant when everything was written down somewhere.

The missing thing wasn't documentation. It was the conversation that produced it. Better documents preserve understanding; they don't create it. The creating happened in the room. When the room went away, the understanding went with it. The documents stayed. But documents can't be asked. Documents can't notice when a technical decision made six weeks ago no longer fits what the system looks like.

And then I came along with an AI power tool and started shipping against that gap at three times the previous pace.

The review bottleneck wasn't really about reviews. It was about shared understanding. Nobody could review fast because nobody had been in the room when the thinking happened, and half the thinking had never been written down at all.

---

Working on JoustMania, the same Bluetooth party game behind the KubeCon talk, we wanted to add a feature that remembered how players had played before. Dynamic behavior that adapted over time. Interesting idea. We implemented it with AI assistance: storage, pipelines, game recording, replay. Each piece followed logically from the previous one. The agent was never wrong about any individual step.

The review request sat open for weeks. We never merged it.

The feature was pointless. JoustMania is a local party game. Controllers get passed around, sessions end, nobody registers. There's no way to know if the same person is holding the same controller from one game to the next. Player history doesn't attach to anything. We'd built a sophisticated solution to a problem that didn't exist.

Nobody asked that question before the first line of code. We asked it, implicitly, by never merging the review request.

The boulder made it all the way up. Then it just stayed there.

---

Addy Osmani, writing about loop engineering in June 2026, quoted Boris Cherny, head of Claude Code at Anthropic: "I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops." Osmani's own observation is that two people can build the same loop and get opposite results: one using it to move faster on work they understand deeply, the other to avoid understanding the work at all. "The loop doesn't know the difference. You do."[7]

The loop amplifies whatever understanding the team brings to it. If the team didn't build shared understanding before the loop ran, the loop executes the misalignment faster. The boulder doesn't just go up faster; it goes up faster in the wrong direction.

Eliyahu Goldratt's Theory of Constraints[8] says one simple thing: optimizing anything that isn't the bottleneck doesn't improve throughput. It just moves work faster toward the constraint. If shared understanding is the bottleneck, if the team's ability to agree on what to build is what limits delivery, then making implementation faster

[7] Addy Osmani, "Loop Engineering" (addyosmani.com/blog/loop-engineering/), quoting Boris Cherny of Anthropic.

[8] Eliyahu M. Goldratt and Jeff Cox, *The Goal* (North River Press, 1984); the Theory of Constraints.

doesn't help. It produces more output against a constraint that hasn't moved.

The boulder goes up faster. The constraint was never the pushing.

Some teams will read this and disagree. They removed the planning sessions, went AI-first, doubled their output, and things are working fine. They might be right. For now. What "for now" means varies: sometimes it's the first time a senior engineer leaves and the system becomes inexplicable to the people who remain. Sometimes it's the first time a feature request surfaces a decision nobody remembered making. Sometimes it's the moment the codebase gets complex enough that the informal knowledge can't hold it together anymore. Speed without shared understanding creates debt that compounds invisibly.

That paragraph is unfalsifiable as written, and I would rather say so than let it stand. If every team that disagrees is simply pre-symptomatic, nothing counts as evidence against me, and an argument that cannot lose is not worth your time. So here is what would settle it: a team runs deliberate sessions for a quarter and watches whether the numbers that matter move. If none of them do, the book was wrong about that team, and the debt I just described was a story I told myself. Which numbers, and how to count them so the answer means something, is Appendix A. I would rather hand you the test at the front than at the back.

Most teams I see have done the same thing I did. They've added AI to the implementation end of the process and called it transformation. It's just acceleration, and acceleration without a better process gets to the wrong place faster.

The instinct is understandable. AI tools drop into an existing workflow cleanly. The engineer opens an editor, starts prompting, ships more code. The feedback loop is immediate and satisfying. The problems it creates are delayed and diffuse: a review queue that grows quietly, a shared understanding that gets thinner as the pace increases, a team that starts to feel like an audience for one person's output.

Changing the process is a different kind of work. It requires the team to agree that something is broken before the pain is obvious. It requires investment in planning before anyone has written a line of code. It requires trusting that the time spent in a room together, before the agent runs, is the most valuable engineering time a team has, and that's a hard sell when the backlog is full and the AI tool is right there.

---

The agent loop is real. AI can take a well-specified ticket, implement it, run a review pass, flag what doesn't fit, and cycle back, without a human in the middle. It's available now, and it works when the input is good enough.

That loop is the agent's: implementation, review, iteration, at machine speed. To its right sits production, where the output meets reality. To its left is where the thinking happens: what to build, why, and what done looks like. This book is an argument for the left of the loop. For the humans who need to be there while the work is still on the page, building the shared understanding that determines whether the loop produces the right thing.

The agent runs the fastest loop anyone has ever had, and there is only one thing in it that can be corrected. What it cannot correct is the spec. Questioning the frame instead of the output is a different loop, an older and slower one, and it still needs the room.[9]

The input is the problem.

OpenAI described this precisely after shipping roughly one million lines of production code with Codex agents: "from the agent's point of view, anything it cannot access in-context effectively does not exist."[10] Knowledge in Slack threads, in documents, in someone's head:

[9] Chris Argyris and Donald Schön drew the distinction in *Theory in Practice: Increasing Professional Effectiveness* (Jossey-Bass, 1974); Argyris made it famous three years later in "Double Loop Learning in Organizations" (*Harvard Business Review*, September 1977). Single-loop learning corrects errors against goals it takes as given. Double-loop learning questions the goals. The agent loop is a single loop by construction, and the fastest one ever built: the goal is the prompt, and nothing inside the loop can reach it. The second loop is what the rest of this book is about.

[10] OpenAI, "Harness Engineering: Leveraging Codex in an Agent-First World" (openai.com/index/harness-engineering/).

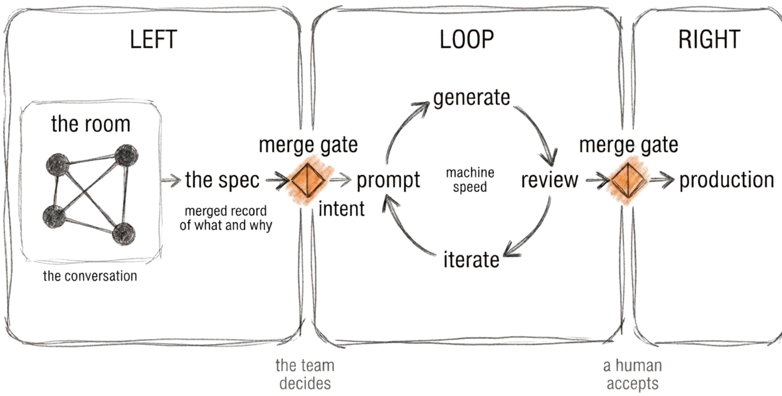


Figure 1: The room merges into a spec; the spec merges into the loop, where the agent generates, reviews, and iterates at machine speed; the loop merges into production.

invisible to the system. The agent builds from what it can see. What it can't see might as well not exist.

Garbage in, garbage out, just faster. If the prompt reflects a thin scoping document that a product manager wrote on a six-month horizon while everyone else was working on tickets, the output will be confidently, efficiently wrong.

The shift has to happen before a line is written. Collaboration moves left: into planning, into the conversation, into the moment where the team builds shared understanding of what they're actually trying to build.

If the team can be in the same room, or on the same call, that's the strongest version: synchronous, whole team, shared understanding built in real time. Everyone leaves with a spec and everyone in the room owns it.

Not every team has that option. When the timezone gap is six hours, synchronous starts to mean someone sacrificing their morning or their evening on a regular basis. For teams in that situation, the planning needs a different form factor. But the outcome has to be the same. A shared decision, made by everyone who needs to act on it, recorded so the agent can build from it.

Think of it like a review request. Not on code, on the spec itself. The spec opens for review. Anyone can comment. Decisions get made in the thread, not in a side conversation between the product manager and two seniors. And there's a merge condition: a moment where the team agrees this is decided, this is what we build from, the agent can run.

That last part is what our Confluence documents were missing. They existed. They sat there. But there was no merge gate. No moment where the team said this is decided. No record of why the decision was made, only what it was. Six months later, when the system has changed and the decision no longer fits, the team needs to know whether to revisit it or discard it. The reasoning is what tells them. A document with no comment history and no approval record gives them the what with none of the why. A spec treated like a review request gives them both.

The agent working from a spec like that is in a different position. It has context, not just requirements. It can flag when a new ticket contradicts a previous decision. It can surface the reasoning when something looks wrong.

The spec becomes a living record of intent, not a snapshot of what one person thought the product should be.

---

I'm not writing this from the outside. I was the power user creating the bottleneck. I was shipping against a ticket I hadn't fully questioned, in a team that hadn't fully owned the thinking behind it, across a timezone gap that made everything slower and more isolated.

The process wasn't broken in an obvious way. It was broken in the quiet way that distributed async teams break: everyone doing their part, nobody in the same moment, understanding siloed in the people who wrote the documents rather than shared across the people doing the work.

What I needed wasn't a better AI tool. I needed the team to converge on intent before I started. In a room if possible. Async if not. Together, deliberately, with a record of what we decided and why.

The process was the problem. But the process isn't the only thing AI changes. It also changes what's left for the people inside it.

# The End of a Craft?

*"Craftsmanship names an enduring, basic human impulse, the desire to do a job well for its own sake."* — Richard Sennett, *The Craftsman* (2008)[11]

*If the agent does the building, what's left that only a person can do?*

Everyone is asking this out loud now and most of the answers are either panic or a sales pitch.

The honest answer is the work that can't be prompted: the judgment before the prompt exists and after the agent has run.[12] The agent is extraordinary at building what it's told to build. It has no opinion about whether that was the right thing to build, or whether the system it's being added to is becoming something coherent or something nobody can hold.

That judgment has a name once it's shared. Direction. And it's the part of the craft the agent doesn't touch.

---

Direction is easy to mistake for vision, which sounds like a slogan on a wall. It's more concrete than that. It's the running answer to "where does this need to go," held closely enough that the next decision can be checked against it. Whether today's choice still makes sense six months from now. Whether the system is becoming one thing or three.

[11] Richard Sennett, *The Craftsman* (Yale University Press, 2008).

[12] This is "Discernment" in the 4D Framework for AI Fluency — Delegation, Description, Discernment, Diligence — developed by Rick Dakan and Joseph Feller with Anthropic (aifluencyframework.org). The framework names the individual competencies for working with AI; this book is about the shared understanding a team builds before those competencies have anything reliable to act on.

The agent has none of it, and not because it isn't capable enough yet. It optimizes the request in front of it, which is what it's for. Asked whether this is a good implementation of what it was told, it will often answer better than a person could. It has nothing of its own to say, though, about whether the system should be heading there at all, because that question is about the whole history of the system and where it's meant to end up. That picture lives in people who have watched the system become what it is.

It's fair to push back that the agent isn't blind to that history anymore. Memory persists across sessions, and context files like AGENTS.md[13] let a team write the picture down and hand it over. That's real, but it moves the work rather than removing it. Someone still has to write that picture, keep it true, and hold it in common. A memory kept in one person's local setup keeps the picture as private as keeping it in their head did: the drift between two copies is now written down, and still invisible to anyone but them. An agent can hold direction. It can't decide it: choosing what's worth building is a judgment about the world outside the context window. It can't notice when the direction goes stale, because staleness is measured against conversations the agent wasn't in. And it can't know whether the copy it holds is the one the team is actually steering by, because that fact lives in the team, not in any file. Three gaps, one shape: direction is a relationship to a team, not a string in a repository.

And more of the picture isn't automatically better. A context swollen with every past decision can drag the agent down the way a session left running too long does, because the bloat buries the signal, and only a person can tell which part is still load-bearing and which has turned to noise. A fresh, well-aimed start often beats a long one carrying everything it ever accumulated.

The judgment shows up before anything gets built. Which approach. What scope is worth it. What "good enough" means for this team, this week. Deciding that is direction work, and it's irreducibly human.

---

[13] AGENTS.md is an open format for agent instruction files (agents.md), adopted across a range of coding agents.

Over one Christmas I built a Rust evaluation engine that compiled to WebAssembly, a small program in a language I'd never written a line of. The same open-source specification had a separate implementation in five languages: Python, Java, .NET, TypeScript, and Rust, and each was quietly making slightly different decisions at the edges. One engine compiled to WebAssembly meant one set of decisions, every language inheriting the same behavior. The agent wrote every line. I couldn't have. What I brought was the judgment around it: that the disagreement between the five was the actual problem to solve, that a single shared engine was the way to end it, and that a rough holiday prototype was enough to prove the idea. The agent built. I decided what was worth building, why, and how we'd know it worked.

> That division is the whole shape of the work now. The agent on implementation. The person on direction.

It was always the more interesting half. The craft was never the typing. It was the judgment the typing crowded out. Implementation was just where the time went. Take the time away, and what's left is the part that took years to learn.

---

Direction has two failure modes, and the agent will flag neither.

The first is that nobody holds it at all. The work still gets done: tickets close, features ship, the agent never tires. But every decision is made against a local picture, and the local pictures slowly stop agreeing. The system becomes one thing in one corner and a different thing in another. Months later someone reaches a choice that no longer fits and goes looking for the reason it was made, and there isn't one, because the person who made it was answering a smaller question at the time. Nothing announces this. It's the most expensive kind of drift exactly because no single step ever looked wrong.

The second is subtler, because it looks like the problem is solved: one person holds the direction, and a direction held by one person is just an opinion. The moment someone else builds against a different picture of where the system is going, the work diverges quietly, because nobody is wrong, they're just pointed differently. A Northstar only does its job when the team is steering by the same one. And a shared direction doesn't come from one person thinking clearly. It comes from a room.

That's the argument the rest of this book has to earn. For now it's enough to say the craft didn't end. One craft did, the implementation most engineers built their identity on. What replaced it is the judgment that implementation used to crowd out.

So the craft survives, moved up to direction, the high, slow plane, where the system is headed over months. But the agent erodes judgment closer to the ground too: in what the tool quietly stops an engineer from noticing, one prompt at a time.

# A Fool with a Tool is Still a Fool

*"A fool with a tool is still a fool."* — Grady Booch (attributed)[14]

*If judgment is the craft that's left, what does it catch that the tool can't?*

My first serious attempt with an AI coding tool was extracting an API package out of a large open source SDK. The goal was clean enough: separate the API from the implementation, two packages instead of one. Standard refactoring work, the kind of thing that should be mechanical once you know what you want.

I burned two days on it. Two days of reshuffling and redefining, no useful progress, the agent confidently helping me go in circles, producing output that looked plausible and went nowhere. I learned a lot. I produced nothing useful.

A week later I tried again. Same task, same tool. It worked.

The difference was that I now understood the system well enough to tell it what I actually needed. I'd failed my way into the right level of specification. I knew what to point out, what to constrain, what to leave open. The prompt was better because my thinking was better.

That's the thing that gets lost in the enthusiasm about AI coding tools. The tool doesn't make the engineer smarter. It makes the engineer faster, but only at the level of thinking they already have.

[14] Widely attributed to Grady Booch; the exact origin is unverified.

If the engineer's understanding is shallow, it produces shallow output quickly. If the specification is vague, it fills the vagueness with confident guesses.

A fool with a tool is still a fool.

---

I got better over time. Prompts that would have taken me two days of burned tokens eventually took two hours. I learned what the tool needed, where it would go wrong, and how much guidance to give it. But that learning was entirely private.

I failed, observed, adjusted, retried. Nobody watched that process. Nobody else on the team went through it with me. I was building the understanding of how to specify, how to constrain, how to prompt for the kind of system we actually needed, and it lived in my head. It didn't transfer. It didn't accumulate anywhere the team could access it.

Which means everyone else who picked up the tool was starting from scratch. Their own two days of burned tokens. Their own private iteration loop. Learning the same lessons in isolation, making the same early mistakes, building the same understanding that never quite made it out of their head either.

AI's quietest effect on teams is the gradual privatization of operational knowledge. How to work with the agent well is hard-won understanding. And by default it goes nowhere.

---

I was getting faster. Tests were thorough. Implementation looked structurally clean. I had developed a review habit. Before handing anything to the team, I'd check my test cases, look for edge cases I might have missed, flag anything that seemed odd in what was being tested.

What I wasn't checking in the same way was the implementation's behavior. The agent wrote it, it passed the tests, it looked structurally sound. My attention had shifted, upward toward architecture and coverage, away from granular behavioral correctness at the line level.

There was a junior engineer on the team. First or second year. Exceptional eye for detail. He started finding things. Regularly, in a pattern, nothing alarming, just consistent. Edge cases. Behavioral inconsistencies. Small gaps between what the code did and what the project specification said it should do.

Nothing catastrophic. But in a feature flag backend, behavioral correctness isn't optional. A flag that evaluates differently than the specification says it should is exactly the kind of bug that's invisible until it isn't.

He was catching things I'd stopped looking for. Not because he was better than me. Because he was still looking at the thing my attention had drifted away from. He read for behavior. I'd started reading for structure, because structure was what the agent handed back to me and what the tests confirmed.

The tool hadn't made me worse. It had reorganized where my expertise got applied. And the reorganization created a blind spot I couldn't see, because I was focused on the output the tool produced, not on what the output wasn't telling me.

That blind spot only became visible because someone else on the team was still looking at the full picture.

What "a fool with a tool is still a fool" actually means isn't that AI makes engineers foolish. Most engineers using these tools are experienced and careful, and they develop real skill with the tool over time. The problem is subtler: the tool quietly shifts what the engineer pays attention to, and the things it shifts them away from don't announce their absence. They go unnoticed until someone else catches them, or until nobody does.

---

The team is the check on what the tool can't see.

It works as a different set of eyes at a different level of the system. The junior was valuable because he was still reading the code the way you read code before you've developed the habit of trusting the agent's output. He was closer to the behavior than I was.

But that check only works if the team is actually looking. And the same forces that push engineers toward private AI tools push them away from collective attention. Everyone heads down with their own context, their own loop, their own hard-won understanding of how to prompt well. The team starts to feel like a set of individuals who share a codebase rather than a group of people who share a system.

The agent only validates against the prompt. If a prompt reflects one person's understanding of what the system should do, that's all it gives back. The team is the only mechanism that puts more than one person's understanding into the spec before the agent runs.

Prompting skill is the smaller part of what matters here. System thinking is the bigger one, and it's what gets built collectively: in the room, in the review, in the moment where someone with a different level of attention says that's not quite right, here's what the spec actually says.

A fair objection here: I just hadn't learned to use the tool yet. The two days of burned tokens were a skill problem, not a fundamental limitation of AI. And that's partly true. I did get better. The tool did get more useful as I understood what it needed. But that's exactly the point. The skill the tool requires didn't come with the tool, because it's the kind of thing that used to develop in the room, and when everyone learns to prompt privately, it develops alone or not at all.

Extreme Programming[15] made this argument in the nineties: understanding develops through collaboration, not in isolation. The industry mostly ignored it in favor of individual productivity metrics and frameworks that optimized the team away.

AI is making the XP argument unavoidable by forcing the team to do what the tool can't.

---

The junior was doing *recognition work*: catching the wrong shape before it compounds. It's a skill, and it doesn't only belong to juniors. An eval checks output against intent someone already encoded; recognition catches drift from intent nobody had encoded yet, before there's a test to write.

It's quiet, constant work. It surfaces in a code review, in exploring an unfamiliar part of the codebase, in the planning pass before a spec

[15] Kent Beck, *Extreme Programming Explained: Embrace Change* — discussed, with citation, in the introduction.

goes to the agent. An engineer reaching for a pattern the team abandoned two years ago. A hand-rolled utility that duplicates what the shared library already provides. None of it is catastrophic. All of it gets more expensive the longer it stays. And none of it is something the agent catches reliably, because it doesn't know what clean looks like in this particular system, accumulated over time. The engineer does, from having been in the system, from having seen the messy version enough times to recognize it on sight. The eye develops through exposure. It can't be specced, and an agent can't be prompted to have it.

What I do with the recognition matters as much as the recognition. Reviewing agent-generated code on the JoustMania codebase, I noticed it had built three separate structures to track one piece of state where a single one would have been clean. It worked. It passed the tests. And it meant every future change would touch three places instead of one.

I didn't fix it. I created a ticket.

Not because the fix was hard. Because fixing it in place would have conflated two things, the work I was doing and the problem I'd noticed, and muddied both. Noticing without immediately fixing, naming a problem without solving it in the wrong moment, is its own discipline. AI makes it harder to practice, because the agent wants to keep going and the flow state feels productive. The pause to write the ticket instead of fixing in place is a small act of system thinking over immediate output.

That discipline catches the wrong shape once the work exists. The wrong ask arrives before it, from outside the room.

# The Product Owner is Dead. Long Live Product Thinking.

*"The king is dead. Long live the king."* — traditional proclamation of royal succession

*If the team can catch the wrong shape, who catches the wrong ask?*

Open source maintainers are drowning. I'm one of them.

The pattern is the same everywhere. Someone opens an issue asking for a quick way to verify a concept. A contributor picks it up, feeds it to an agent, and a pull request arrives. Thorough. Tested. Sometimes even clean. Sophisticated, extensible, production-ready. And wrong at the level of need. The maintainer now has to review the generated work, explain why it's the wrong level of solution, and do it without discouraging the contributor from trying again.

Multiply that by twenty open pull requests.

Open source is just a distributed async team with an unstable roster and no onboarding. It always has been. Contributors come and go. The maintainers are the only constant. There's no planning, no shared context that builds over time, no Product Owner holding the thread between handoffs.

I maintain OpenFeature, a feature flagging specification where behavioral correctness matters. A contributor once opened a large pull request, too large to review properly, so I asked them to split it into two for a better overview and an easier follow.

They came back with two pull requests, split by module. I'd meant by feature. The agent had also duplicated the shared logic between them instead of keeping it in one place. Technically compliant with my request. Structurally worse than what we started with.

They hadn't been careless. The agent hadn't malfunctioned. The instruction, split into two pull requests, was given without the context that would have made it meaningful. The agent filled the gap with the most obvious interpretation of my words, not my intent. What was missing was *minimum viable context*[16]: enough to rule out the obvious wrong reading. The agent did what it was told.

What followed was frustrating for both of us. A lot of back and forth. Tension that shouldn't have been necessary between two people who both wanted the right outcome. We got there eventually: two pull requests, one based on the other, a stacked diff that worked. But the path to it cost time, energy, and goodwill that a shared understanding of the problem upfront would have saved entirely.

Nobody validated the need or the approach before the agent ran.

What did I change after this? Honestly, not much, and that's worth saying directly. CONTRIBUTING.md gets ignored. Issue templates get skipped. In open source, a maintainer often can't have that conversation before a contribution arrives. The async distributed nature of it means the shared understanding has to be front-loaded somewhere else.

The best partial mitigation I've found is writing issues with more context: minimum viable context in the issue description itself, so the contributor, human or agent, has less to guess at. Not the full spec. Just enough to make the obvious wrong interpretation less obvious.

It doesn't prevent the problem. It makes it cheaper. And in open source, that's often the best available answer.

---

[16] The name leans on minimum viable product; the discipline leans on the minimal reproducible example that open source and Stack Overflow have demanded for decades. Same instinct, aimed at understanding instead of bug reports.

In product teams the same chain exists, though the Curse of Sisyphus chapter's product manager and this chapter's Product Owner (PO) are different roles that land in the same seat, not one person renamed. A stakeholder has a need. A PO interprets it. The team builds the interpretation. Somewhere in that chain the actual need gets distorted through the normal compression that happens at every handoff. By the time it surfaces, something is already built.

The cost is rework. Delayed delivery. A PO who has to go back to a stakeholder and explain why what was built isn't what they meant. That conversation is never easy. And it keeps happening.

The titles vary and the industry has never agreed on the boundary. This chapter is about whoever sits between the stakeholders and the team, carrying both the product thinking and the organizational interface.

The PO exists, in part, to manage that chain. To be the translation layer between what stakeholders want and what teams build. To carry the context that doesn't fit in a ticket. To field the questions that surface mid-implementation when something doesn't quite add up.

It's a structural role for a structural problem and the role works until it becomes the structure's failure point.

The role doesn't have an off switch. Everything that can't be handled elsewhere routes through the same person. Stakeholder pressure from above. Requirement gaps from below. Product decisions that have to be made before anyone knows enough to make them confidently. POs I've known carried it well, until the structure broke them. Not because they were weak. Because it was set up to eventually break anyone.

---

Implementation got cheap. Discovering it was wrong didn't.

In the open source example, the maintainer pays that cost. In a product team, the PO pays it first, then the team pays it in rework, then the stakeholder pays it in delayed delivery. The misunderstanding moves through the system and someone absorbs it at every stage.

Collapsing the implementation phase surfaces misunderstandings faster. The agent runs, the output exists, the gap between what was needed and what was built becomes visible sooner. That's good if the spec was right. It's expensive if it wasn't.

A spec written before implementation starts and reviewed by the stakeholder catches the misunderstanding at the cheapest possible moment. The stakeholder reads it and says "that's not what I meant." Nothing has been built yet. The clarification costs a comment in a document, not two weeks of rework.

I do a lighter version of this without a spec at all. When a PO brings a ticket, I ask what the motivation behind it is: the need the ticket is standing in for. Often that need is smaller than the ticket. It could have been solved with something simpler, but the ticket had already been engineered into a full solution before anyone checked whether the need required one. The same wrong level as the open source contribution: a thorough answer to a question nobody validated. The question catches it one ticket at a time, before the build.

In the open source world that's a fast experiment before a sophisticated solution. In a product team it's a spec review before implementation starts, a review request on the spec instead of the code. The form is different. The principle is the same: validate the need before the thing gets built.

---

The traditional PO carries two things simultaneously. Product thinking: challenging intent, representing user needs, asking why before the team builds what. And stakeholder management: navigating organizational pressure, translating business context into something a team can act on, carrying the political weight to push back when the request is wrong.

Those are different skills bundled into one role partly by necessity. Building the spec together changes the necessity.

When the team builds the spec together, when product thinking happens collectively, that first part starts to distribute. Senior engineers develop the habit of asking why before how. Tech leads challenge intent as part of the planning process, not as an afterthought. Product thinking becomes a broader team competency over time, rather than something concentrated in a single role.

That doesn't happen immediately. Some teams need a dedicated PO because that muscle isn't there yet. It becomes a development goal, not a permanent structure.

The second part doesn't distribute. Stakeholder navigation requires years of context that most engineers don't have and don't want to develop: the organizational interface, the political skill of managing what comes into the team from outside, the credibility to push back on an executive request. It's what survives as a dedicated role.

I did a version of this as a team captain. Requests came down from above, strange ones, impractical ones, and I made the call on them: some we didn't pursue, some we implemented differently than asked. As long as we honored the defined interface, the implementation was ours. That judgment, deciding what to absorb, what to push back on, what to reshape, is the part that doesn't distribute. It's not product thinking, and it's not something a team picks up in a planning session.

Something more like a Stakeholder Navigator, not a Product Owner in the agile sense. The person who gets the spec in front of the right people at the right moment, manages the review cycle, and protects the team from the noise while they do the thinking.

I know how that sounds. A rebranded PO. Partly yes. The difference is ownership. The old model has the PO owning product thinking and managing stakeholders. This one has the team owning product thinking and the navigator owning the organizational interface. That's a real split even if the job title looks similar from the outside. And it's a more sustainable job, more defined, with an off switch.

---

The open source maintainer and the PO are dealing with the same structural problem from different angles.

The spec review is the mechanism that closes the gap between what's asked and what's needed. A defined moment where the need gets validated before anyone builds anything: a checkpoint, not a planning ceremony. Where the stakeholder can say "that's not what I meant" when it's still cheap to hear it.

Birgitta Böckeler,[17] analyzing the harness engineering practices that surround agent systems, names the limitation precisely: the harness can govern how code gets written without ever checking that it does what users actually need. Technical correctness and functional intent are different problems. The harness solves the first. The spec review solves the second.

The role evolution is a consequence of that. When the spec carries the shared understanding, when the team built it together and the stakeholder saw it before the build, the translation layer the PO was providing becomes less necessary. What remains is the person who managed the organizational interface well enough to get the spec in front of the right people in the first place.

That person still matters, maybe more than before, and they just don't have to carry everything anymore.

Roles shift and responsibilities redistribute: the product thinking spreads into the team, and the organizational interface narrows to the one role that protects the room while the team does the thinking. All of it assumes the room exists. A place where the thinking actually happens, and a team that knows how to use it. That's the thing nobody has built yet.

---

[17] Birgitta Böckeler, "Harness Engineering for Coding Agent Users," in the "Exploring Gen AI" series (martinfowler.com/articles/exploring-gen-ai/harness-engineering.html). "Harness engineering" is a shared term. OpenAI uses it too (the Curse of Sisyphus chapter), and Böckeler's earlier memo on it opens by saying she had just read their write-up, so this is one term spreading rather than two people coining it.

# The Agora

*The agora was the heart of the ancient Greek city, a public space where citizens gathered not just to trade, but to think, argue, and decide together. It was where the polis became more than a collection of individuals.*

*If no one person owns the understanding, how does a team build it together?*

Every team has a place where the real thinking happens. Sometimes it's a meeting. More often it's a corridor conversation, a Slack thread, a one-on-one where someone finally said the thing they'd been sitting on for a week. The technical lead and the product manager talking through a problem before it becomes a ticket. The senior engineer pulling a junior aside to explain why the approach won't work. The offhand comment in a code review that reframes the whole feature. That thinking is the most valuable work the team does, and even in well-documented teams, part of it stays invisible, dependent on the right people being in the right conversation.

The Spec Session is where a team makes that space intentional and the whole team converges on one spec before the agent runs. XP called it the planning game.[18] What's different is the stakes: where XP had an individual coder, the Spec Session has an agent. And the quality of the shared thinking before the agent runs determines everything that follows.

If this sounds like Scrum or Kanban done right, that's because it is. The activity is old, and the baggage on the old names is real, but neither is the reason for a new one. What changed stands at the end of the session: an implementer that never gets stuck, and never walks

[18] Kent Beck, *Extreme Programming Explained: Embrace Change* — discussed, with citation, in the introduction; "the planning game" is one of its practices.

over to ask. Sprint planning done right was enough, for the coder who could. The Spec Session is the same convergence, run for the one that can't.

The individual coder was doing a second job nobody ever named. A human who doesn't understand what they're building gets uncomfortable, or stuck, and walks over to a desk and asks what was actually meant. That discomfort was a safety net nobody scheduled: it caught planning failures in week two instead of production. An agent has no such habit. It runs with its best interpretation or it stops, and neither one walks over. It can ask a question, but it asks the person who wrote the prompt, from inside the framing that prompt set. The old process could afford an ambiguous spec because the implementer and the circuit breaker were the same person. The agent split them apart, and the Spec Session is what replaces the walk to the desk: the same catch, moved to the cheapest moment.

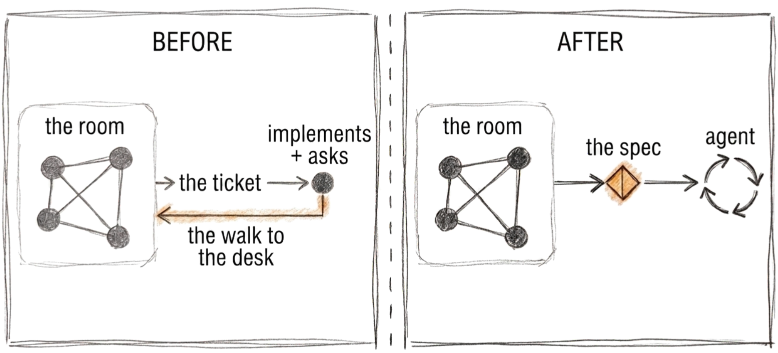


Figure 2: Before: the room hands off a ticket, and the implementer walks back to ask what it means. After: the room converges on a spec, and the agent runs from that directly.

A better agent closes some of this. One that notices the ambiguity and asks the room instead of the author would restore the walk to the desk, and that would be worth having. It would still leave the understanding where it was answered, with one person. The session exists because the team has to hold it, not because the agent has to receive it.

---

Before the Spec Session starts, there needs to be a reason to build anything at all. The business need, the user problem, and the organizational imperative matter equally. The Spec Session aligns the team on what gets built and how, not whether it's worth building. Without a clear need, the best-run Spec Session produces a beautifully shared understanding of the wrong thing. This is a dedicated space and not a meeting: meetings have agendas, action items, owners. A Spec Session produces something different: a room full of people who hold the same picture of what's being built, why, and what done looks like. The document is the record of that convergence. It doesn't need everyone to read it the same way, only to keep pointing at the same thing.[19]

That's why the path is the goal. A team could be handed a perfectly written spec and skip the session. They'd own none of it, having inherited someone else's thinking rather than built their own. And when the agent runs and something unexpected surfaces, the team that built the understanding together knows what to do. The team that received the spec has to go find the person who wrote it.

---

I run climbing courses for children. At the start of every session we sit in a circle: everyone speaks, everyone listens, no rushing past. We reflect on the last time. We let the room breathe. We create a shared awareness of where everyone is before anyone starts climbing.[20]

[19] The sociological term is a boundary object — an artifact "plastic enough to adapt to local needs ... yet robust enough to maintain a common identity across sites" (Susan Leigh Star and James Griesemer, "Institutional Ecology, 'Translations' and Boundary Objects," *Social Studies of Science*, 1989).

[20] Developmental psychology has a name for this: shared intentionality, the capacity for joint attention and joint goals that Michael Tomasello identifies as the distinctively human cognitive trick, present in infants before language (*Why We Cooperate*, MIT Press, 2009). Joint commitments carry norms: Gräfenhain, Behne, Carpenter, and Tomasello showed that even three-year-olds take leave before abandoning a joint activity rather than walking away silently ("Young Children's Understanding of Joint Commitments," *Developmental Psychology* 45, 2009).

Afterwards, the peer learning emerges. The child who felt heard speaks up on the wall. The one who listened offers help without being asked. The group becomes something that supports itself.

When we skip the circle, whether because there's no time, the energy is off, or someone rushes past it, the session feels disconnected. Individuals climbing in parallel rather than a group climbing together. The difference is subtle, real, and almost impossible to point to for anyone who hasn't felt it.

Software teams need the same thing, as a disposition the team carries continuously: the habit of making thinking visible before acting on it, of listening before building, of checking where everyone is before assuming the team already knows. The climbing circle is one way to make that instinct visible. Some teams do it in how they write issues, or in the way a senior pauses before running the agent to ask whether the spec is clear. The form doesn't matter, but the instinct does. That's the circle, before the agent runs.

The Spec Session is not a planning gate or waterfall with better tooling. The team still iterates. Still puts things in front of users. Still discovers that what got built isn't what was needed. The difference is that the team iterates with everyone holding the same picture, so when the direction turns out to be wrong, everyone understands why, and the correction is faster. In complex domains, where cause and effect only become visible in retrospect, the Spec Session doesn't replace iteration; it makes iteration less expensive by ensuring the team is wrong together.

Iterating is often the better move, and it is worth being exact about when. A probe beats a room when three things hold: the loop is short, the cost of being wrong is bounded, and the wrongness is legible when it arrives. Under those conditions building the thing is the cheapest way to learn what it should have been, and running a session first is overhead.

The room earns its place when the third condition fails. JoustMania's player history got built in full, storage and pipelines and recording and replay, and building it taught us nothing, because the feature was never wrong in its output. It was wrong about what it recorded. A controller ID is not a person, and no number of runs surfaces that. One question does.

Iteration surfaces wrong answers. It does not surface wrong questions. A build that answers the wrong question passes its user test and solves nothing. Which is also when being wrong together is worth something: reality corrects a wrong answer for everyone at once, and leaves a wrong question standing.

What that condition builds is more than shared understanding. It makes trust possible.

In climbing, trust has a physical form. Belaying, holding the rope that keeps another person safe, is the most literal expression of it. One person's life, in a meaningful sense, in another's hands. In my children's courses, we build to that point deliberately. The circle first. The shared awareness. The environment where everyone feels safe enough to be uncertain out loud. And then something that still surprises me happens. Five-year-olds belay each other. Supervised, of course, but the trust is real, the technique is real, and the care they take with each other is genuine. I have watched five-year-olds belay more attentively than adults I've seen on real rock, because the environment built the trust before the task required it. That's the standard software teams are measured against.

How many engineering teams have the trust required to say "I don't understand this spec"? To tell a senior their approach is wrong? To admit mid-implementation that something doesn't make sense and risk looking slow?

The hardest part is building this culture, regardless of the format.

A Spec Session breaks when people don't say what they actually think: the junior who spots a problem but doesn't feel safe raising it, the engineer who thinks the approach is over-engineered but defers to the senior, the concern that never gets voiced because the culture doesn't make it safe.

Psychological safety[21] is the prerequisite for a Spec Session. Without it, the team has a meeting where people perform agreement and the real thinking still happens in corridors afterward.

[21] Amy Edmondson, "Psychological Safety and Learning Behavior in Work Teams," *Administrative Science Quarterly* (1999); later popularized via Google's Project Aristotle research.

The Spec Session makes that culture structural rather than personal. It says: this is the time, this is the room, this is the moment where it's not just safe but expected to say what you think. The format protects the behavior that good leads were trying to create informally.

Spec Sessions can become the new endless ceremony when the team circles the problem, surfaces every concern, explores every edge case, and never converges. Three hours later there's a rich document and no decision.

The antidote is a rotating session lead: someone whose job is to get to convergence, and to close the discussion when the understanding is good enough. Their job is also to protect the session from becoming the thing it was designed to replace.

---

The Spec Session produces minimum viable context: the agent runs without guessing, the team reviews without re-learning, the stakeholder recognizes what they asked for when they see it.

The session needs to surface three things before it closes.

What are we actually building. The specific behavior, the boundary conditions, the thing that would make a skeptic say "yes, that's done." The team needs to be able to describe it without looking at a document.

Why this approach and not another. Not for the audit trail, but for the moment six months from now when the system has changed and someone needs to know whether to revisit this or discard it. What was built goes obsolete. Why it was built is what that decision runs on.

What we're not building. The explicit scope boundary. The thing that's out of scope this iteration, that will be a different ticket. Without this, the agent fills the silence with guesses that are reasonable and still wrong.

When those three things are in the room and have been shared, challenged, and confirmed, the session is done.

The spec that comes out is not a user story wearing a new name, and it isn't acceptance criteria either. A story was a placeholder for a conversation that would happen during implementation, because the implementer could ask; the spec is the record of a conversation that already happened, because the implementer can't. Acceptance criteria are a contract, not an understanding.

The difference fits on half a page. What I actually typed to that OpenFeature contributor: this pull request is too large to review properly, split it into two for a better overview and an easier follow. What a session would have recorded instead: two pull requests, split by feature, not by module, each reviewable on its own. The shared logic stays in one place; neither duplicates it. Why: a better overview, and a follow that doesn't require holding two diffs in your head at once. Not doing: no restructuring of the existing module layout, no new abstractions for the shared code. Done when each pull request reviews cleanly by itself and the shared logic exists exactly once.

That's one task's record. Yours will look different, because your gaps are different.

Consensus is too high a bar and too slow to reach. The Spec Session produces consent, the decision-making distinction sociocracy has used for decades.[22] Nobody objects strongly enough to block, and that's enough. The team can move with a shared picture even when individuals would have made different choices. The document captures them. The team owns them. The agent can run.

---

The best room I have ever worked in had no walls at all.

Async Spec Planning is the same thing with a different form factor: for distributed teams, the session becomes a review request on the spec, and the merge condition is that the team has converged. It's slower than a room and faster than a document that gets written, inherited, and silently misunderstood.

But the async form carries a risk the room doesn't. What stops one developer from using an agent to write the spec proposal, and another from using an agent to review it? If both sides of the conversation are AI-mediated, the human understanding loop is bypassed

[22] Consent rather than consensus is a core distinction in sociocracy, formalized by Gerard Endenburg in the Sociocratic Circle-Organisation Method in the early 1970s and carried into Sociocracy 3.0 (sociocracy30.org).

before the code loop even starts. The spec merges. The understanding was never built. A merged spec that collected silent agreement is a session that never happened; if the understanding lives only in the artifact, the session failed.[23]

The full form, its mechanics and its mitigations, is Appendix D.

---

Tools are beginning to encode this instinct.[24] Birgitta Böckeler,[25] a Distinguished Engineer at ThoughtWorks, has analyzed the leading ones and identified the central gap: all of them assume a single developer does the requirements analysis. None address what happens when multiple people need to arrive at the same picture together. AWS's AI-DLC is the one exception, and it puts a room back: a ritual called mob elaboration, cross functional and hours long, before any agent runs.[26] Then it bets everything downstream on the artifacts that room produced, which is the bet this book argues against. A tool can encode the instinct, but it can't replace the room.[27]

So start small. Smaller, even, than the team this book will argue for.

Grab a coworker before your next task. Define the plan together. Define the prompt together. Then hand it to the agent and let it run against what you agreed. If you want to change something mid-run, stop: that's another spec, not an edit to this one.

[23] Chad Fowler's Deletion Test poses the same question at the artifact layer — imagine deleting the implementation, then ask whether the knowledge survives or the code was the only place it lived. Same structure, applied to code instead of a spec document. Chad Fowler, "The Deletion Test," *The Phoenix Architecture* (aicoding.leaflet.pub).

[24] A snapshot of that landscape, and where this book sits in it, is Appendix E.

[25] Birgitta Böckeler, "Understanding Spec-Driven Development: Kiro, Spec-Kit, and Tessl," "Exploring Gen AI" (martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html). She analyzed Kiro, spec-kit, and Tessl.

[26] Raja SP, "AI-Driven Development Life Cycle: Reimagining Software Engineering," AWS DevOps & Developer Productivity Blog (aws.amazon.com/blogs/devops/ai-driven-development-life-cycle, 2025), and the AI-DLC Method Definition paper linked from it. The method assumes synchronous colocation — "a single room with a shared screen" — and persists every artifact it produces, from intent to test plan, as traceable context the AI references across the lifecycle.

[27] Simon Martinelli's *Spec-Driven Development: From Specs to Code with AI Agents* (Apress, 2026) is a book about the artifact, and it stops at the same line. The requirements workshop produces "a shared mental model that cannot be produced by tools alone," and "AI does not replace this human collaboration. Its role starts after the workshop." His life cycle then runs specify, generate, validate, review, refine, and the workshop is not one of the five. Where the two books part is in Appendix E.

Let the output tell you whether the spec was good enough.

If it worked, iterate from there. You don't need to change your whole process from one day to the next.

If it didn't, ask why together. Where did the agent take the wrong turn? Would a better explanation have prevented it? Did it fill a gap you didn't know you'd left? Did you hand it so much context that the signal drowned? There are many ways a run can fail, and you and your coworker are the only ones who can tell which one you hit. A failed run is the cheapest teaching material you'll ever get, but only if the lesson lands in both heads. A lesson learned alone is the problem this book is about, in miniature.

Figure out what works, and build on it. One experiment at a time, one task at a time. That's the thing engineers are good at anyway: fixing broken things.

A fuller starting shape waits in the working template in Appendix B.

---

The Spec Session is where the work happens. Which leaves one question worth sitting with: why does it have to be people in the room at all?

# The Ever-Agreeing Genie

*Kent Beck calls his AI pair a genie: it grants wishes without judgment, exactly what you asked for, whether or not it is what you needed.*[28] *The danger was never the genie. It was the wish.*

*If a team builds understanding together, what keeps it honest?*

I'm building a demo for my next talk on JoustMania again: OpenTelemetry and OpenFeature wired together into what I'm calling an agentic nervous system, a game that runs its own experiments over time and tracks what happened.

I got it working and watched it run.

And right behind the working demo came the doubt. Is this worth building, or does a framework already exist that does this better? Observability data is what the system already tells you about itself while it runs, so is it actually a sound foundation for the agent's experiments, or a convenient one I've talked myself into? I don't know which. The talk isn't even accepted yet. Nobody has seen it.

I research the answers with AI, and I notice what that research doesn't give me. It's fluent inside the architecture I hand it: ask whether the design is sound and it reasons about the design I described. Ask whether something already solves this and it searches for the thing I named, not the thing I didn't know to look for.

So I have a system that works, and a talk that might never get a stage. Either way, I still don't know if the shape of it is right.

[28] Kent Beck named the metaphor first. See "Genie Wants to Leap," *Tidy First?* (tidyfirst.substack.com/p/genie-wants-to-leap).

---

There are four things you don't get from an AI, four things only another person can provide.

The first is *weighted validation*. When someone with their own hard-won experience, their own perspective, their own reasons to disagree, looks at what you're building and says "yes, that's right," it changes something, because they're independent of your framing. Their agreement costs them something: they considered it and chose to agree.

Someone who has built observability pipelines telling me the foundation holds would settle something the model's agreement can't.

AI agreement is cheap. Human agreement from the right person is not.

The second is scope judgment. Am I reaching too high or not high enough? Is an agentic nervous system too ambitious for a conference demo, or not ambitious enough to deserve a stage? I can't see that from inside it. An AI will help a person execute whatever direction they've chosen. That takes someone who has seen both failure modes: the plan that collapsed under its own weight and the one that never got anywhere, and can recognize, from the outside, which one the work is heading toward.

The agent will hold any direction. It can't tell you yours is wrong.

The third is the unknown unknown. The tool no one in the room is aware of. The approach that would make everything simpler but hasn't been encountered. If a framework already does what I built over the demo, the model and I are both searching for a name I don't know.

AI can surface unknown unknowns within the space it has been pointed at. Ask it "what am I missing here" and it will often find something useful. The harder gap is context-specific: the thing someone with direct experience of the situation would see that neither you nor the model would think to name. That requires someone looking in from outside the framing.

You can't prompt for what you don't know to name.

The fourth is teaching. This one has no JoustMania version. There is a difference between a thing being explained and a thing being taught. AI explains well. It can walk a person through a concept, surface the relevant literature, answer follow-up questions patiently. But teaching requires someone who recognizes when the learner doesn't know something yet and can hold them through that state until the understanding arrives. The scar tissue matters, and so does the presence of someone who notices they're there and knows what to do about it.

Research is a substitute for teaching, slower and lonelier.

---

The Spec Session is where the first three happen, before the agent runs.

The distributed async team loses these things first. The timezone gap, the Kanban board, the scoping documents that arrive as finished artifacts, all of it slowly drains the moments where weighted validation happens, where someone challenges the scope, where an unknown unknown gets surfaced by someone who happened to know the thing no one else did.

And AI accelerates the drain through usefulness. The tool is faster, always available, never in a meeting, so the maker stops reaching for the person and starts reaching for the tool.

Until you have a working demo and no stage to test it against, and the question of whether the shape is right sits with you in a way it wouldn't if someone else had checked the architecture first.

---

Another person can provide all four, but a person isn't a structure. The structural answer is the team.

This isn't only the solo builder's problem. The same gap opened on a team a few chapters back, where the junior catching what the agent and I had both stopped seeing was supplying exactly this, a check against reality that no prompt produced. Take that junior out of the room, let everyone review alone against their own prompt, and the agreement gets cheap again.

The team is the environment where independent judgment can happen, the place where judgment gets tested. Where someone pushes back from a starting point that isn't yours, the one place an agent, working from your framing, can never stand.

AI is frictionless by design. It finds a way to make an instinct seem reasonable. It smooths the edges off the direction already chosen. That's useful for execution and dangerous for judgment.

The team has friction by nature. Different experience, different exposure, different mental models of where the system is going and why. That friction is the mechanism. The reason the team's judgment is different from the AI's agreement is precisely that the team doesn't have to agree.

A form of friction can be engineered from an AI: build the context to argue back, load it with counterarguments, instruct it to push back. But the pushback is still downstream of the maker's framing. The AI can challenge stated assumptions. The unstated ones are invisible to it.[29]

Anthropic's own harness engineers supply the strongest version of the counterargument: a planner, a generator, and a skeptical evaluator, converging on a spec before anything runs.[30] Scope challenged, validation adversarial by design, and a human in the loop reading the result; the room, the argument goes, automated down to one reader. Even the argument doesn't claim teaching.

Here is what the harness does and doesn't do. It produces a better artifact, and the claim was never about the artifact. The evaluator has nothing to lose by saying no, so its validation carries no weight in the only sense that matters.

---

[29] Alibaba's Qwen team makes the same point formally in "The Verification Horizon: No Silver Bullet for Coding Agent Rewards" (2026, arxiv.org/abs/2606.26300): every verifier is only a proxy for human intent, never the intent itself.

[30] Agents grading their own work reliably skew positive, and making a generator critical of its own output proved far less tractable than tuning a separate, skeptical evaluator, whose judgment, in turn, had to be calibrated against a human's. Notably, the harness also converged on a spec-first ritual: a planner scoping the work, then generator and evaluator negotiating what "done" looks like before any code is written. Prithvi Rajasekaran, "Harness Design for Long-Running Application Development" (anthropic.com/engineering, 24 March 2026).

Three roles, and that number is worth taking seriously. Without reading a word of this book, the harness converged on the same minimum a room needs to check itself.

But a planner, a generator, and their evaluator are still a converged trio, not three independent rooms. Each has to be tuned differently, or it grades itself generously. However differently they're tuned, all three are handed the same framing. Any of them can challenge the stated assumptions; the unstated ones are as invisible to all three as to one, and for the same reason. Questioning a framing takes a standpoint outside it, and there is no outside inside a context window.

Which leaves the human reading the converged result, and the Agora already named that seat: a team handed a perfectly written spec owns none of it, having inherited someone else's thinking. The harness is the strongest tool yet for writing the record. The theory still lives where it always did.

There is a name for what a convergence ritual invites, and the reader who knows it has been waiting for it: groupthink. Three people in a room can lock onto a shared error faster than one alone, because now the error has social endorsement.

The symmetry is worth naming, since it was just used against someone else's system: a room is handed one framing too, and the assumptions nobody in it states are invisible to all three for the same reason they're invisible to the planner, the generator, and the evaluator. If that objection lands against the harness, it lands here.

No control makes a room immune to it, but its convergence is never total: three people carrying three different decades that predate the team, not one shared tuning. Its composition can change too, a junior added, a lead rotated, someone pulled in from outside the domain. The trio's spread is bounded by how it was tuned; the room's is bounded by who gets let in, which is why consent instead of consensus and a lead who rotates exist in the first place, not to make the room immune, but because rooms converge.

The alternative to imperfect controls is not immunity either: a developer alone with an agent is the most converged group of all, a groupthink of one with an amplifier. The room at least contains controls. The desk contains none.

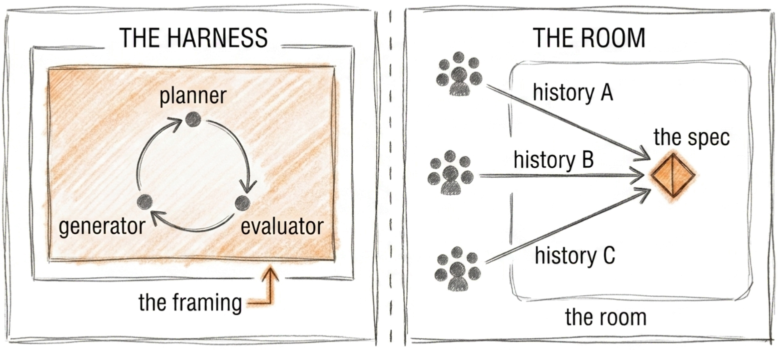


Figure 3: The harness's planner, generator, and evaluator are handed one framing; the room's three people bring three histories to the same spec.

None of the four is a capability gap. Capability gaps close; that's what model releases do. Wanting takes two things a release doesn't ship: a standpoint outside your framing, and something to lose by using it. Weighted validation, scope judgment, and the unknown unknown all need the first, and this chapter has already watched two attempts fail to manufacture it, the engineered pushback and the converged trio, both still working downstream of the maker's framing. Weighted validation needs the second too: an evaluator tuned to withhold agreement pays nothing for the withholding, while a person who says no carries the consequence of having said it. That's why the human doesn't move out of the loop as the model improves. We were never in it because we were better at something. We're in it because the loop is for us.

Understanding is only useful if it can be challenged. And the place where it gets challenged, by someone who doesn't have to agree, is the team.

The friction is the point.

Three of the four keep your judgment honest while you build. Teaching does something the other three don't. It is how the judgment survives the person who has it, and it is the first thing the agent takes. So where does the next person learn to be the one who doesn't have to agree, when the tool is always the faster answer?

# The Alexandria Problem

*The Library of Alexandria was the ancient world's greatest repository of knowledge, and its greatest single point of failure. It didn't burn in a single night. It declined across decades, through neglect, through the slow erosion of what it had been. By the time it was gone, almost no one had noticed it going.*

*If the understanding holds today, how does it survive the people who leave?*

I learned to be an engineer in code reviews, and in the pairing sessions and planning meetings around them.

From watching seniors think out loud, never from documentation or courses: how they searched for the problem beneath the problem, how they argued about tradeoffs, how they asked questions that reframed everything before anyone had looked at a single line of code.[31]

The most important thing I learned was how to ask the right question. How to formulate a problem, and the motivation behind it, so clearly that the solution could emerge from it, not knowing the answer, but knowing how to construct the context that makes the answer findable.

I learned that most deeply in open source. Suddenly the person reading the issue knows nothing. They weren't in the meeting. They

[31] This is Peter Naur's claim from "Programming as Theory Building" — cited, in full, in the introduction. Naur held that a new programmer can only acquire the theory of a program through active engagement with the people who already hold it, and that the theory itself cannot be written down. He drew the conclusion at maximum strength: a program whose team has left should be discarded and rewritten, because the theory died with the room.

don't know the system, the constraints, the previous decisions. I have to build the entire picture from scratch: structure the problem so a stranger can understand it, provide every piece of context someone outside my head would need to engage with it meaningfully.

That's a harder discipline than explaining something to a colleague who shares the same background. A colleague fills in the gaps from shared experience. A stranger doesn't. Open source teaches the contributor to make the problem legible to someone with no obligation to figure out what they meant.

But there's a second discipline open source teaches that's equally important. Not over-sharing.

As little as possible, as much as needed. Too little context and the stranger can't engage. Too much and the contributor has made their problem the stranger's problem. They have to sort through noise to find what actually matters. The discipline is knowing what to leave out: what's assumed, what's load-bearing, what would change the answer if it were different.

It's the same discipline as writing a good prompt. Too much context and the agent loses focus, chases the noise, produces something that addresses everything it was told and misses what was meant. Too little and it fills the gaps with confident guesses. The discipline runs past the single prompt: knowing when to clear a session that has run too long and start fresh, scoped to the real problem, is part of the same skill.

There's a failure mode minimum viable context is designed to prevent. The XY problem.[32] Someone has problem X, thinks the solution is Y, asks for help with Y, and the person helping them never finds out about X. The answer to Y lands, it doesn't solve anything, and everyone wasted time.

It's endemic in open source. Someone asks a specific question about a specific implementation detail, and three comments in a maintainer asks "what are you actually trying to do," and the real problem surfaces. Which has a completely different, usually simpler solution.

[32] Coined by Mark Jason Dominus in a 2001 comp.lang.perl.misc post; popularized since via xyproblem.info.

It's equally endemic in AI prompting. The engineer prompts for Y, the agent implements Y confidently, and X remains unsolved. The agent doesn't ask what they're actually trying to do. It assumes Y is the right question because they asked it.

Minimum viable context means surfacing X before asking about Y. It means providing the right context, not just enough of it. The actual problem, not the assumed solution.

That's what the senior in the code review was doing with "what are you actually trying to solve here": checking for the XY problem before engaging with the implementation.

---

When I watch junior engineers struggle with AI tools today, it's almost never a technical problem. It's a problem-formulation problem.

They're bad at structuring the question, not because they're bad engineers but because they haven't yet developed the habit. They go straight to the agent with a vague sense of what they want. The agent produces something. They take the output and keep going.

The senior in a code review would have stopped them before that. "What are you actually trying to solve here?" Asked enough times, by enough seniors, in enough reviews, that question is how they internalize it. How they start asking it unprompted before anyone else has to.

Without that environment, the junior keeps getting output. Just not always the right output. And they don't yet have the experience to tell the difference between a good answer to a bad question and a good answer to the right one.

The AI doesn't teach that distinction. It's too helpful. It tries regardless of how vague the input is. It doesn't say "your question is too vague." That's useful once they know how to evaluate the output. It's a trap when they don't yet know what good looks like.

---

Mob programming works because everyone is in the thinking together. The navigator holds the direction. The driver holds the keyboard. Everyone else watches, questions, catches what the driver missed, builds the shared understanding that makes the session worth more than the code it produces. The output is almost secondary. The point is what happens in the room while it gets made.

Then someone, not the navigator, just someone, started prompting on their own machine. Through efficiency, not malice. The agent was faster. A solution arrived. The group looked at it. We moved on.

The mob didn't break dramatically. Nobody announced a change of approach. Someone just quietly did it differently, the output appeared, and the group followed the path of least resistance. The session continued. The mob stopped. The fun went with it, and so did the learning. Knowledge that used to build through watching each other reason, through the disagreement and the question and the "wait, what about this edge case," stopped accumulating the moment the output arrived pre-formed.

We were still in the room together. We had stopped thinking together.

That's the Alexandria Problem: not an event but a process. The library doesn't burn in one dramatic fire. It burns one session at a time, one private prompt at a time, one skipped conversation at a time. The knowledge that used to accumulate through proximity and repetition stops accumulating the moment the friction disappears.

And AI removes the friction so efficiently that no one notices the burning until the smoke is already gone.

It's the Codex problem from the first chapter, seen from the inside this time. What never leaves someone's head is exactly what the next agent can't reach, and a private prompt is how it stays there.

It happened in code reviews, in pair programming, in the hallway conversation where a senior explained why the approach was wrong while the work was still an idea.

AI tools, used individually, break that transfer. Just by being available. The junior doesn't need to ask the senior. The agent is faster and less intimidating. The senior doesn't need to explain; they can just do it themselves.

The moment of friction that used to produce a teaching interaction gets resolved by the tool before it becomes a conversation.[33]

The junior develops their own AI workflow in isolation, without the senior's eye on the process, and what they're not developing is the underlying skill: the problem formulation, the question that focuses the work before a prompt is written.

---

The Spec Session is the new learning environment, arrived at by consequence rather than design.

When the team is in the room together, building the spec before the agent runs, the junior watches how seniors frame problems. They see the questions that get asked before anyone proposes a solution. They see an experienced engineer push back on a direction not because the implementation is wrong but because the problem statement isn't tight enough. They see what "ready for the agent" looks like, and more importantly, what it doesn't look like yet. They learn it from watching humans think through what needs to be generated and why.

The mob exploration works the same way. A new library, a new approach, a PoC the team runs together. The junior isn't just evaluating the technology. They're watching how experienced engineers evaluate tradeoffs. What questions they ask. What concerns they raise. What "good enough for now" means versus "this will hurt us later."

It runs both ways. We were evaluating high-availability tooling once: paid caching, Redis, the works. A junior asked why we didn't just poll and persist to a volume on the cluster. We'd been too deep in the sophisticated option to see the plain one. He dragged us out. He was learning how the team weighed tradeoffs by sitting in the room, and

[33] Anthropic put numbers on this in a randomized controlled trial: junior engineers learning a new library with AI assistance scored 17% lower on comprehension minutes later — nearly two letter grades — with the largest gap in debugging, while the speed gain was not statistically significant. How they used the tool decided the outcome: those who delegated generation or leaned on AI to debug scored lowest; those who asked conceptual questions or interrogated the generated code preserved their learning, and the conceptual-inquiry group, errors and all, was among the fastest. The authors note the study used a chat assistant, not an agentic tool, and expect agentic impacts to be more pronounced. Judy Hanwen Shen and Alex Tamkin, "How AI Impacts Skill Formation" (Anthropic, January 2026; arXiv:2601.20245).

from outside the hole we'd dug, he asked the question we'd stopped asking.

You can't get that from a spec document or from reading the agent's output. None of this is formal teaching; nobody stands at a whiteboard. Experienced engineers make their thinking visible, and the junior learns how the answer gets found.

If the team stops being the place where that transfer happens, everyone works alone with their own tools, and the next generation develops faster in some ways and not at all in the one that matters most.

What builds that judgment is a career spent in rooms where people think out loud, ask hard questions, and show the junior what careful problem formulation looks like by doing it in front of them.

Seniors develop prompting habits nobody else sees. Architects discover patterns nobody else inherits. Knowledge that used to become team knowledge remains personal knowledge, and there's no code review to surface it, no pairing session to transfer it, no hallway conversation to pass it on. The junior feels the impact first, but the loss belongs to the whole team.

When the senior's attention shifts upward, from implementation to architecture, the junior is often left downstream, catching the behavioral edge cases the senior no longer has the bandwidth to notice. That division of attention can work, but only if it's deliberate.

The senior whose attention has shifted to the architectural plane has a new responsibility in the Spec Session: to pull the junior up with them. To ask the questions that make architectural thinking visible: why this approach, what happens at scale, what would break this in six months. And to ask them not just in private, but in the room, with the junior present and expected to engage. Otherwise the junior develops a different and narrower skill: valuable, but not the same as learning to think architecturally.

The skill that makes a great engineer is the one thing the agent can't teach, because it's learned by watching a human do it. If the room goes quiet, the next generation inherits the tools without the judgment that makes the tools worth having.

But if the room is where that judgment transfers, how does anyone know the transfer is working? How does anyone know the understanding is actually shared and not just assumed?

# The Oracle

*The Oracle at Delphi was the most authoritative voice in the ancient world. Kings and generals sought its guidance before every major decision. The oracle always answered. The answers were always true. And they were almost always misread.*

*If you can't see understanding, how do you know it's there?*

I was once told it made me look bad that I hadn't posted anything in the demo channel.

My direct lead pointed it out. He wasn't seeing my contribution and was questioning whether I was contributing at all.

What I had been working on was a package-private, domain-driven setup for a Java service: clean encapsulation, limited visibility of methods and classes, as little side effect as possible. The kind of structural decision that makes everything built on top of it easier, safer, and more maintainable. The bedrock that lets features ship reliably without unexpected interactions. The foundation: nothing shippable, nothing to demo.

I nodded and ignored it. The measurement seemed wrong and I knew the work was right. My role was eliminated shortly after. I've never been able to fully separate the two. When a metric doesn't see someone's work, the people reading the metric don't see it either.

What did I do after being told? I pushed back. I didn't start posting in the demo channel. I informed the team about what I'd been told, questioned it openly, and named it as a bad metric, in the same category as review request count, lines of code, or tokens burned.

Metrics that measure activity rather than value, and that get gamed the moment they become targets.

The metric couldn't see it. And when I think about what might have changed the outcome, it wasn't doing different work. It was being asked a different question. Not "why aren't you in the demo channel" but "what are you working on." The first has a measurable answer. The second requires a conversation.

The metric was visibility. The work was invisible. The oracle gave a true signal, and nobody thought to question the reading. Questioning it is a more useful act than changing your behavior to satisfy it.

Teams have always rewarded the visible over the valuable. The feature over the refactor. The new thing over the stable thing. The demo over the test suite.

But AI makes it catastrophic. Because now the demo channel can be full every day without much effort. Ship a feature, open a review request, close a ticket. The numbers look incredible. And the invisible work, meaning the quality, the structure, the foundation, gets starved of attention because it doesn't register anywhere that matters.

The person doing the invisible work gets told it makes them look bad. Or loses their role. Or quietly stops doing the work that nobody can see.

---

We used to confuse effort with value. AI lets us confuse output with value.

DORA's 2024 research on generative AI[34] put numbers on this. For every 25% increase in AI adoption, local process metrics improved: code quality, documentation, speed of code reviews. The same increase correlated with a 1.5% drop in overall delivery throughput and a 7.2% drop in delivery stability, and developers reported

[34] DORA, *Impact of Generative AI in Software Development* (dora.dev/research/ai/gen-ai-report), the generative-AI companion to the *Accelerate State of DevOps Report 2024*: a 1.5% reduction in delivery throughput and a 7.2% reduction in delivery stability for every 25% increase in AI adoption, estimated associations, not causal claims.

higher flow and personal satisfaction while spending less time on what DORA, that year, categorized as valuable work.

These were correlations, and DORA was careful to say so. The explanation it offered was a hypothesis, not a finding: AI lets a team produce far more code in the same time, changesets grow, and bigger changes have always been slower to land and quicker to break. The time saved got lost downstream: in toil, in reviews that couldn't keep pace, in merges too large to be safe. The oracle measured the right things at the wrong level.

The dividend arrives at the task level and the system can't absorb it. DORA's 2024 numbers ran the wrong direction for anyone who already believed implementation was abundant. DORA's 2025 report is the flip: throughput and time on valuable work both reversed positive.[35] What didn't reverse is instability, still climbing on the far side of that flip, and the difference is everything upstream of the agent: the thin spec, the wrong thing built quickly, the rework nobody counts.

METR supplies a different kind of evidence, not a dividend that failed to land, but one that wasn't there: experienced developers ran 19% slower on repositories they knew well and came away certain they'd gotten faster,[36] the same failure to collect showing up as perception instead of throughput, at the scale of one developer instead of a team. A throughput gain a team cannot land safely is not a dividend it has cashed, and until the process lets it cash, the room costs principal. Appendix A's quarter is the test, and it names the measures. Flat after a quarter of full sessions, and the premise doesn't hold for that team.

---

One more reading, and it's the one I like least. The evidence I have for implementation getting cheap is my own: an engine I could not have

[35] DORA, *State of AI-assisted Software Development* (dora.dev, September 2025): the throughput relationship reversed, with AI adoption now associated with increased throughput, improved product performance, and more time on valuable work, while delivery instability continued to increase.

[36] METR, *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity* (metr.org, July 2025; arXiv:2507.09089). Sixteen experienced developers were 19% slower with AI on repositories they knew well, yet believed it had made them roughly 20% faster.

written, a refactor that went from two days to two hours, a pace the team could not absorb. All of it self-reported, and the METR trial in this chapter is precisely a finding that self-reports about AI speed are unreliable, in the direction I would want them to be wrong. Sixteen developers believed they were 20% faster and were 19% slower. I have no reason to think I'm exempt. So I should stop making the claim I can't support. Whether AI made me faster at finishing work is exactly the thing METR says I am the worst available witness to, and I am not going to argue my way around a study I put in this chapter myself.

That was never the claim this book needs. What it needs is smaller and harder to dispute: producing something that looks finished, and that somebody now has to read, got radically cheaper. That one leaves marks outside my head. Review requests open at once. A queue that stopped draining. A feature built across a handful of evenings that would have taken a month of them. None of it depends on my sense of how fast I was working.

METR's developers were slower and certain they were faster, and the reason that illusion is so convincing is that the output really was arriving. Their finding and mine are the same finding read from opposite ends. What got cheap was the artifact, not the outcome, and a team's process is calibrated to the rate at which work arrives needing attention rather than the rate at which it gets finished. Only the first one moved. That gap is the subject of this book.

---

The metrics we use to measure engineering output were wrong before AI. AI just made them easier to game and faster to break.

Lines of code. Story points. Number of review requests. Tokens burned. Each measures something real in isolation.

Goodhart's Law[37]: when a measure becomes a target, it ceases to be a good measure. A metric stops describing what matters and starts describing the behavior it incentivizes.

[37] After the economist Charles Goodhart (1975); the familiar phrasing is Marilyn Strathern's (1997): "When a measure becomes a target, it ceases to be a good measure."

An AI power user can inflate every one of those metrics simultaneously. Lines of code go up. The agent writes more than a human would. Story points get closed faster. The agent implements tickets in hours. Review requests multiply. Why open one when five will do. Tokens burned increase, and that's just the cost of running the agent.

And the dashboard looks great. Leadership sees throughput.

The review queue is growing underneath it. The specs are getting thinner because there's no time to write them properly when the agent is already running. The codebase is accumulating decisions that made sense in isolation and don't fit together. The team is heads down, individually productive, collectively drifting.

---

These are all activity metrics, counted per person, a choice the numbers never required.

They were questionable even then. But individuals were the unit of delivery: one engineer owned a feature end to end, designing it, building it, testing it, shipping it, and measuring the individual at least pointed at the output.

In a team-delivered, agent-executed workflow, the individual is no longer the right unit. The agent handles implementation. The value comes from what the team understood and agreed on before the agent ran: the spec, the shared context, the collective judgment about what to build and why. None of that shows up in an individual's metrics.

Worse, individual metrics actively work against shared understanding. If story points closed is how an engineer gets recognized, the incentive is to close tickets, not to spend time in a spec session building context that benefits the whole team. If review requests opened is the signal, the incentive is to keep the agent running, not to slow down and question whether the spec is right.

The usual response when this becomes visible: add another skill to the agent. Improve the prompt. Use a better model. Find a way to make the AI produce better output with less review.

The output isn't the problem. The process that produces it is. Adding capability to the agent doesn't fix a broken spec; it produces better-written code against the wrong intent, faster. If that counts as a fix, the incentive has been made more powerful.

---

The metric that actually matters is predictability.

Can the team tell a stakeholder when something will be done, and then actually do it by then?

That's the signal that the process is working. That the specs are good enough to commit from. That the team understands the work before it starts. That surprises are rare because the thinking happened upfront.

Predictability doesn't require the work to be foreseeable. In domains where answers only show up by building, the team changes what it promises, not whether it keeps promises: by Thursday we'll know whether approach A holds, instead of the feature ships Thursday. Shaping the commitment to what the team can actually know is itself understanding, made visible to the people outside the room.

A team with high velocity and low predictability is guessing fast. A team with lower velocity and high predictability has shared understanding. The second team is more valuable, more trustworthy, and more capable of scaling, because when capacity is added to a predictable process, the predictability holds. When AI is added to a broken process, the breakage accelerates.

Predictability is also the metric that can't be easily gamed. Either it was called right or it wasn't. No amount of demo channel activity changes whether the thing shipped when the team said it would. No number of tokens burned makes a forecast accurate.

And predictability is a team metric. One person can't be predictable alone. Predictability requires shared understanding of the work, shared ownership of the sizing, shared commitment to what goes through the gate. It requires the Spec Session, or something like it: a moment where the team agrees on what is being built and how big it is, while the work is still on the page.

No points, no poker. Estimation was how teams committed when implementation capacity was the constraint, and the constraint moved. What sizing looks like instead is in Appendix B. What the room costs, honestly counted, is Appendix A.

None of this makes predictability immune to gaming. A team can pad the estimate, promise six weeks for work that takes three, and hit the date every time. But that only holds as long as nobody outside the team asks why three-week features take six, and stakeholders eventually do. Sandbagging predictability takes the whole team agreeing to the same lie and holding it quarter after quarter. Inflating throughput takes one person and an agent for an afternoon.

Teams have always known that upfront alignment produces better estimates, and that's not a new idea. What's new is that AI makes the alternative, skipping the alignment, running the agent, seeing what happens, feel so productive in the moment that the incentive to do the upfront work disappears.

The demo channel fills up. The forecast gets noisier. From the outside, everything looks great.

---

Output is easy to buy now. A thousand tokens spent implementing the wrong thing creates exactly the same value as a thousand hours spent implementing the wrong thing: none.

The first team looks unstoppable on paper: four times the review volume of a year ago, an active demo channel, a healthy burndown chart. Underneath, reviews are backing up, commitments slip, stakeholders have stopped trusting the estimates, and rework is quietly eating the gains.

The second team ships less visibly, but when they say something will be ready on Thursday, it's ready on Thursday. Reviews are fast because the specs were tight. Stakeholders plan around their commitments.

The first team is optimizing for output. The second team is optimizing for understanding.

From a dashboard, the first team looks more productive. From a planning meeting, the second team is more valuable.

Reliable forecasts tell a manager something deeper than delivery dates. They say the team shares a model of the system, the problem, and the solution. The alignment was real, and the forecast is the evidence.

What to measure is whether the team can say what they'll produce, and when, and whether they're right.

The teams that turn understanding into reliable commitments are the ones creating value. Everything else is just numbers getting easier to game.

Predictability shows that the team understands the work. What it doesn't show is how that understanding gets built.

# The Trireme

*The trireme's rowers sat in three banks at three different heights. The lowest worked their oars through leather sleeves close to the waterline and saw almost nothing of where the ship was going. It was the fastest warship in the ancient world because all three ranks pulled to one rhythm.*

*If predictability is the proof, what kind of team produces it?*

---

Team sizes have always been a function of the bottleneck. In traditional agile, 7 to 10 people was the recommended unit, calibrated to sustain an implementation pipeline. The number was the answer to a specific question: how many people does it take to keep implementation moving?

Agents change the answer, because they change the question. The Spec Session is now the bottleneck. It gets slower as the session fills up and convergence takes longer to build a shared picture.

So the logic that produced 7-to-10 person teams no longer applies. The right team size is now the one that can run an effective Spec Session.

---

The Spec Session needs three kinds of attention on the work.

Someone who holds the system direction, the direction work from the End of a Craft chapter, now named as a role: the person who tracks what the system is becoming and whether the decisions being made are consistent with that, regardless of title or seniority. In a small team this is often the same person who runs the Spec Session. But there needs to be someone in every team.

Someone who watches the ground. The behavioral edge cases, the implementation details, the things that drift when nobody is looking. This is often where juniors do their most valuable work. The junior catching what the senior's attention has moved past, the dynamic from the Fool with a Tool chapter, is by design.

Someone who manages the outside. Stakeholder pressure, organizational noise, the requests that arrive before anyone has had time to spec them. The Stakeholder Navigator from the Product Owner is Dead chapter. With that role filled, the room stays protected long enough to think.

The agent is the fourth, running only after the other three have done their work.

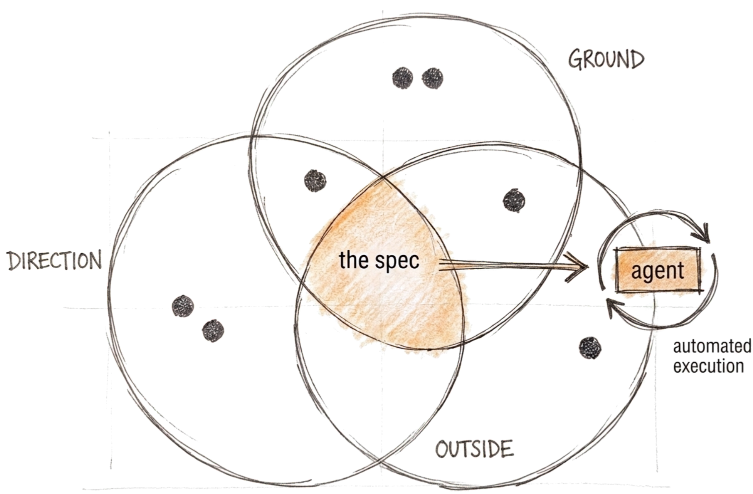


Figure 4: Direction, ground, and outside overlap into the spec; the agent runs only from that overlap.

I've watched these functions show up without anyone assigning them.

In OpenFeature, a contributor once noticed something the maintainers had stopped seeing. Certain providers broke in a subtle way when reused, because they relied on something the code never said. If you knew the assumption, everything worked. The code didn't carry the assumption. The people did.

The cheap fix was obvious. Write it down. A documentation change, a warning in the README.

We were close to taking it. Then one of the maintainers pushed back. An implicit contract that lives in documentation is a contract that fails the moment someone builds without reading it, and someone always builds without reading it. He argued for an explicit interface instead. More work now, less archaeology forever.

None of this happens unless someone accepts the issue first, looking at a report from outside the project and deciding it deserves the project's attention. That sounds like housekeeping. It's the seat where the project meets everyone who uses it and will never show up in person.

Three kinds of attention, and nobody assigned any of them. Someone with eyes on the ground, catching what familiarity had made invisible. Someone answering to the people outside the room. Someone holding direction, thinking past this pull request into the years after it. The structure of the project just makes them show up.

And underneath them, redundancy. The fix didn't go through on one person's say-so. Other maintainers weighed in, pushed on the interface argument, approved it. No single mental model decided the outcome, and no single departure could have taken the understanding with it.

So how many people does that take? Not one per function: the seats overlap, and one person can hold two. As many functions as the problem requires, as few people as the work can sustain. Two people can build a shared understanding. What a pair can't build is a view of itself. Three creates the triangle, the space where a third perspective surfaces what two people, heads down together, stop seeing.

None of this is against pairing, or against the pair the Agora's door opens with: one task, two people, is how you learn the shape, and a pair executing an agreed direction was never the risk. The floor is about the standing practice, and it is older than software. Sociology has known since 1908 that two and three are different structures, not different sizes.[38] The epistemic version is this book's: a pair has no one who can see the framing from outside it. The third seat isn't extra capacity. It's the first seat from which the pair can be seen at all.

Resilience reaches the same number by a different argument. One person holding critical system knowledge is a single point of failure. Two creates a dependency. Three creates a floor that survives someone going heads down, someone leaving, someone getting pulled onto something else.

The minimum viable unit is three people.

---

Still, some organizations will operate at 15 or 20 people, for good reasons: scale, domain breadth, regulatory complexity. What changes is how those teams structure themselves internally.

The model that works is stream-aligned teams[39]: smaller units, each running their own Spec Session on their slice of the problem, each holding deep shared understanding within their domain. Subject matter experts who own their territory. The larger team doesn't need a single room where everyone understands everything. It needs enough boundary knowledge between units that handoffs don't lose context.

Across the whole, what everyone shares is the overall goal, the direction, and the constraints, not the implementation detail of every domain. That shared picture is wider and shallower, answering different questions at different scales.

---

[38] Georg Simmel, *Soziologie* (1908); in English, *The Sociology of Georg Simmel*, trans. Kurt H. Wolff (Free Press, 1950), the chapters on the quantitative determination of the group: no majority, no mediating third, no structure that survives either member's departure until the third seat exists. Simmel's third party mediates conflict and interest; applying the structure to shared understanding and blind spots is this book's extension.

[39] Matthew Skelton and Manuel Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow* (IT Revolution, 2019).

As teams grow larger, agents multiply, and output speeds up, the gap between what the system is doing and what the team collectively understands grows, and the book doesn't pretend to have closed it. Spec Sessions narrow it. Re-alignment across domains narrows it. A platform that makes the invisible visible narrows it. None of them remove it, because the pressure toward speed and the pressure toward drift are the same force under two names.

It's worth being exact about where this model holds and where it stops. The three-role unit contains drift inside a domain: someone holds the direction, someone watches the ground, someone manages the outside, and the picture stays shared because the room is small enough to keep it shared. What it doesn't contain on its own is drift between domains: the gap that opens when ten of those rooms each stay aligned internally and slowly stop aligning with each other. No structure closes that gap by itself. It stays closed only as long as the stream-aligned teams keep choosing to re-share what they've learned, often enough, and early enough, that the divergence never compounds.

That between-domains problem is the layer where Team Topologies operates. Skelton and Pais, whose vocabulary this chapter borrows for its own domain units, map how teams should be shaped and interact at scale, so cognitive load never exceeds what a team can carry: the inter-team question of how the rooms hand off to each other. This book sits one layer down, on the understanding a single room has to build before the agent runs. Team Topologies assumes the work is already understood and optimizes the handoffs; this book argues the understanding has to be built first, and that it takes at least these three functions in the room together. The two are adjacent, not competing; cognitive load exceeding capacity and shared understanding failing to form are the same symptom seen from two layers: teams drifting, decisions made in isolation, systems nobody fully holds.

That's the honest limit. The structure buys aligned domains, not an aligned organization. The only thing that buys the second is the thing the book has been pointing at all along: a room people actually use, repeated at every level that needs one. What has to hold is the room, and the connections between rooms. Whether they exist, and whether people use them, decides everything else.

# The Forest and the Desert

*Beth Andres-Beck and Kent Beck describe two working environments, the Forest and the Desert. The desert runs on the assumption that there is never enough, not enough time, not enough trust, not enough slack, so everyone moves fast and alone. The forest runs on the assumption of enough, and so its paths are shared, and learning accumulates in the undergrowth.*[40]

*If the right structure holds understanding, what keeps it alive over time?*

Kent Beck built Extreme Programming[41] on the conviction that software development is a social activity, that the team is not overhead but the mechanism through which good software gets made. XP was largely set aside in favor of frameworks that separated thinking from building, planning from implementation, people from each other.

AI is the most powerful desert tool ever built. A developer can move faster alone with an agent than any 10x engineer[42] could move alone before, and the tool confirms the desert's founding assumption, that there is no time for the team and no longer any need for one. The desert has never been more accessible, or more tempting. Which makes the forest more important than it's ever been, and more fragile.

---

[40] The forest-and-desert metaphor is Beth Andres-Beck and Kent Beck's. See "Forest & Desert," *Tidy First?* (tidyfirst.substack.com/p/forest-and-desert), and Martin Fowler, "Forest And Desert" (martinfowler.com/bliki/ForestAndDesert.html).

[41] Kent Beck, *Extreme Programming Explained: Embrace Change*, discussed with citation in the introduction.

[42] "10x engineer" traces to Sackman, Erikson, and Grant's 1968 study on individual programmer productivity variance, popularized in software culture via Steve McConnell's *Code Complete*.

The Trireme gave the structure. But the structure depends on a precondition it can't supply: a room where uncertainty is discussable.

The Agora chapter called psychological safety the prerequisite. The sharper version is *epistemic safety*: the conditions where people reveal what they don't know. Where the junior asks the question without calculating the risk first. Where the senior shares the reasoning, not just the conclusion. Where the half-formed thought gets voiced instead of suppressed. Where assumptions become visible before they become decisions.

That room is the precondition for everything the book has argued. Without it there is no shared understanding, no weighted validation, no constructive friction. The XY problem goes uncaught, and the next generation doesn't grow.

The process creates the container; the safe room is what fills it.

---

What breaks the room is rarely dramatic. The subtler damage comes from well-intentioned behavior that lands wrong.

The engineer who questions everything in a review request. Rigorous, thorough, evidence-based. From the inside it feels like raising the standard. From the outside it feels like being interrogated. The junior stops raising concerns because last time it became a debate. The senior stops sharing uncertain thinking because uncertain thinking gets challenged. The room gets quieter. Safer-feeling on the surface. More fragile underneath.

Or the 10x engineer who is genuinely fast, genuinely capable, and experiences the team as an obstacle. Optimized for the desert. They've been rewarded for individual speed their whole career. The Spec Session feels like overhead. The review process feels like a bottleneck. The team feels like it's slowing them down, because it is, because that's partly what the team is for.

I was somewhere on that spectrum. Not hoarding knowledge or dismissing others, but asking too many questions in the wrong format, ending messages with exclamation marks out of habit that read as aggression rather than enthusiasm, and once, memorably, sending colleagues links to blog posts explaining why direct messages destroy flow and that we should stick to processes.

I was right about the substance. Direct messages do interrupt flow. The blog posts probably made good arguments. But sending links to colleagues to explain why their behavior was wrong, however valid the point, reads as condescending. The message was about process. The impact was about respect. I knew it had landed wrong as soon as I sent it. I apologized. The escalation came anyway.

The room was telling me the room was breaking, and not gently.

---

The ego that breaks the room isn't usually the ego that knows it's an ego.

It's the engineer who cares deeply, has strong opinions, asks hard questions, and doesn't yet know how the questions land. The desert virtues, namely rigorous, fast, evidence-based, and process-oriented, are genuinely good engineering habits. They just work differently in the forest.

In the desert, questioning everything in a review request optimizes for correctness. In the forest, it optimizes for silence. The question that makes someone feel tested produces a room where people stop volunteering the uncomfortable truth. And the uncomfortable truth is exactly what the room needs.

AI accelerates this dynamic. The engineer already inclined toward the desert gets a tool that makes it so productive the team starts to feel like friction.

Previously, leverage meant individual output. The 10x engineer was the one who wrote code fastest, shipped most, closed the most tickets. That definition made sense when implementation was the bottleneck.

Until the spec was wrong. Until the direction was off. Until the edge case surfaced after the agent had run three times. Until the junior who would have caught it in the room had stopped speaking up.

When the agent handles implementation, the bottleneck moves. The constraint becomes the quality of the understanding before the agent runs: the spec, the shared context, the room where the direction got challenged before anyone built anything. The engineer who improves the quality of that room now has more leverage than the engineer who writes code fastest.

The 10x engineer of the AI era is the person who makes the room better. Who asks the question that surfaces the XY problem. Who creates the conditions where the junior speaks up. Who slows down the Spec Session just enough to catch the misunderstanding before it becomes rework.

---

A counterargument worth taking seriously: some of the best software ever written came from individuals or tiny teams. Linus Torvalds. Wozniak. The early internet. The forest mostly means walking paths with everyone else, and sometimes it means hacking your own trail, and sometimes the trail is exactly the right call. What made those examples work is that the trails led somewhere others could follow. The desert begins where one person holds the full picture and assumes nobody else needs to. The question is what happens when the system gets complex enough that no single person can hold it, when the edge cases live in the gap between what one person knows and what their colleague knows. That's where the forest earns its place: by surviving what the desert can't.

I've developed a small habit at conferences and events. When standing in a group, never close the circle completely. Leave a gap. Physical space that says: this conversation isn't finished, you're welcome to join. No invitation required. Just an open space where someone could step in.

It's the same instinct as switching to English the moment I know there's someone nearby who isn't a German speaker. Not because they can't understand, often they can. But because engagement is easier when the barrier is lower. The appreciation shows immediately, even from people who would have followed in German. Something relaxes. The conversation opens.

The desert closes ranks: tight clusters, known quantities, optimized for the people already in the room, because sharing costs time it assumes it doesn't have. The forest leaves space before it's requested. It can afford to, because it assumes there is enough to go around, and it accepts the friction of the unexpected perspective because the unexpected perspective is often the one that matters.

The Spec Session works the same way. The session that starts with "we've thought about this and here's what we're doing" has closed the circle. The one that starts with "we have some thinking but we haven't decided" leaves the gap. The junior's edge case, the stakeholder's clarification, the concern nobody voiced in the corridor: those arrive through the gap.

The gesture is small. The effect compounds.

Building the room is slower than moving alone. It requires attention to how things land, not just whether they're correct. It requires making thinking visible even when visible thinking feels inefficient.

It requires, sometimes, someone telling an engineer that their blog post links were not well received.

A team with a safe room and moderate individual capability will outperform a collection of 10x engineers who broke the room, because the room is where the capability compounds. Where the junior's edge case catches the senior's blind spot. Where the Spec Session surfaces the misunderstanding before the agent burns the budget. Where the different perspective changes the direction before it's too late to change.

---

Set the controls beside each other and they are one apparatus. Consent instead of consensus, so agreement is never performed as unanimity. A rotating lead whose job is closing, not winning. The third seat, the first one from which the pair can be seen at all. And the mirror-law: a team that always agrees is not converging, it is reflecting.

Two things hold at any size. A team needs friction, because mirrors don't catch mistakes. And a team needs the room: without people saying what they actually think, nothing else holds.

The desert produces output. The forest produces learning. And the learning is the cost that stays quiet, which is why the desert always looks cheaper than it is.

The forest grows slowly but survives what the desert cannot.

# The Astrolabe

*The astrolabe was the instrument that made celestial navigation possible: a tool for calculating position from stars that were always there but previously unreadable. Before it, sailors estimated. After it, they could know.*

*If one team can sustain it, how does it scale to many without thinning?*

Most developers are running their own agent.

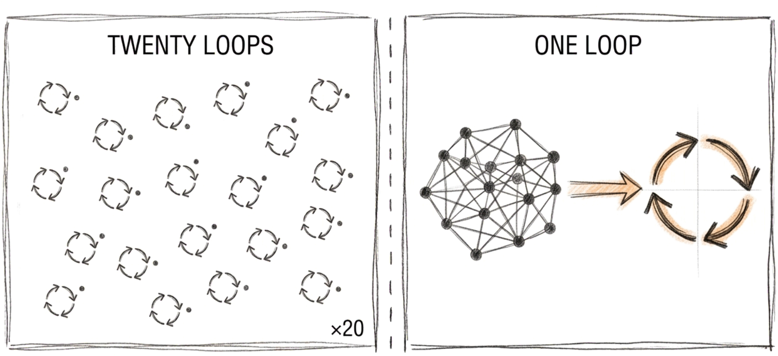


Figure 5: Twenty developers, twenty separate agent loops, versus one team running a single shared loop.

Not every team, every individual. Local machine. Local workflow. Local configuration. Local prompting habits built up through private experimentation that nobody else sees. The AGENTS.md[43] file, if it exists at all, was written by whoever set up the project and hasn't been touched since. The skills and MCPs each engineer uses reflect their own discovery process alone.

A twenty-person engineering organization doesn't have four or five different agent configurations. It has twenty. The fragmentation is at the individual level, which makes it harder to see, harder to fix, and harder to improve.

This is where software teams were before platform engineering existed. Before anyone decided that deployment pipelines, observability stacks, and infrastructure configuration were too important to leave to individual teams with different standards and no shared visibility. Every team managed their own servers. Improvements stayed local and knowledge didn't transfer. The cost of doing it badly was paid independently by each team, invisibly, with no way to see the pattern across the organization.

The same dynamic is playing out now with AI infrastructure. And the solution is probably the same one.

The cost starts when an engineer runs the agent against a thin spec and gets the wrong output. They rerun it. Better prompt this time, or different framing, or more context. The agent runs again. Maybe it works. Maybe it runs again.

That cost, in tokens, in time, in engineering attention, goes nowhere. No ticket captures it. No metric tracks it. The AI budget is treated like electricity: a fixed cost of doing business.

Which means nobody knows which prompts are expensive. Nobody knows which rewrites were caused by bad specs. Nobody knows whether the token spend last month produced shipped value or produced rework. The cost is real and growing; the visibility is zero.

This is the measurement problem from the Oracle chapter, one layer deeper. Nobody can measure what they can't see. And right now, in most organizations, almost nothing about AI spend is visible at the level where decisions get made.

---

[43] AGENTS.md is discussed, with citation, in the End of a Craft chapter.

---

A centralized agent infrastructure changes that.

The platform won't be the only way to run an agent. Engineers will still experiment locally, the same way they still have local terminals despite CI/CD existing. The platform becomes the production path and the source of truth, not the sole execution environment.

But when agent work runs through the platform, something changes. Every run is traceable to a team, a ticket, a spec. Rerun rates become visible. The correlation between spec quality and expensive implementations becomes measurable. The organization can see, for the first time, what a bad prompt actually costs, then trace it back to the process failure that caused it.

That's organizational telemetry for AI development. The same way a team instruments its system to understand its behavior, it instruments its agent platform to understand its process.

Historically, platform engineering worked not because individual teams couldn't build those things themselves. It worked because the platform team sees across every team at once: more signal, more patterns, more ability to spot what's working and what isn't, and the mandate and resources to act on it. One engineer figures out a better way to frame a class of problem. Under the current model that insight stays private, and when the engineer leaves it goes with them. Under the platform model it gets contributed, reviewed, and inherited by every team.

The same logic applies to agent infrastructure. An individual team maintains their own agent configuration with local data and limited resources. A dedicated platform team, agent platform engineers effectively, observes token usage patterns, rerun rates, and prompt failure modes across the whole organization. They spot the skill that's producing consistently poor output and fix it centrally.

The improvement that would have lived in one person's AGENTS.md file becomes the new default for the whole organization. One better skill, inherited by every team. One fixed prompt failure mode, gone everywhere at once. One shared MCP that three teams were building independently, now maintained centrally and available to all.

That's the rate at which the organization gets smarter about working with agents. The platform is valuable not because it saves money but because discoveries stop dying locally.

The foundation is closed to ad hoc modification: stable, trusted, and maintained by people whose job it is to make it better. The team-specific layer is open, customizable, and extensible without breaking what the foundation provides. Teams that find something better contribute it upstream. That is the model open source has run on for decades.[44]

Closing both layers is the failure worth naming, because it looks like success from the center: one configuration, one workflow, one set of prompts, nothing to audit. It also removes what this chapter is measuring for. A team that cannot change how it works cannot discover anything the platform does not already know, and the outlier signal goes quiet, not because nothing is wrong but because nothing can vary.

The platform reaches the agent's memory too. The End of a Craft chapter warned about the version kept in one person's local setup, where the direction persisted but only for them, and any drift from a colleague's copy was written down where nobody else could see it. Held in the platform instead, that inverts: one shared picture, versioned and visible, handed to every agent in the organization. Someone still has to keep it true, but now it's one picture, in the open, where drift shows up instead of hiding.

None of that is simple to build. Shared memory is the hard version, where the moment many agents and many people write to the same store, what's one person's, what's the team's, what's global, and who reconciles two entries that disagree all become real questions. They're the platform team's to own, because it's too big for any one engineer to carry alone.

[44]This is the open/closed principle, one of the five SOLID principles: open for extension, closed for modification. It originated with Bertrand Meyer (*Object-Oriented Software Construction*, 1988) and was popularized as part of SOLID by Robert C. Martin. Applied here to platform architecture rather than to object design. Meyer's concern was dependency: a module others already rely on should not have to be edited to be extended. Read at the scale of an organization rather than a class, that becomes a question about who is allowed to change what, which is the part this chapter uses.

Operational knowledge that used to burn in Alexandria gets written into stone through infrastructure: the living artifact that encodes what the organization has learned about working with agents.

But the platform only solves one layer of the Alexandria Problem. The deeper knowledge, meaning system understanding, architectural context, the reasoning behind past decisions, the edge cases discovered through years in the codebase, still lives in people. The platform can't capture it. The Spec Session is still the mechanism that keeps it shared and alive. The platform makes the operational foundation stable enough that the team's cognitive energy goes toward that deeper work instead of reinventing local agent configurations.

Right now every engineer is simultaneously trying to be productive with AI tools and develop their own understanding of how to use them well. That's a significant overhead that experienced engineers absorb at the cost of other things, and that junior engineers struggle with because they're still building the foundation underneath it.

A platform removes that overhead from the individual and places it with the team best equipped to handle it. Engineers focus on the spec, the problem, the system thinking: the work that requires their expertise and judgment. The platform handles the agent configuration, the skill maintenance, the model selection, the infrastructure for running agent work reliably at scale.

---

A bad spec produces a rerun, and the rerun traces back to the spec, the spec to the team and the ticket. The cost shows up in the data.

It's a feedback loop. For the first time, the organization can test whether the Spec Session matters.

Does shared understanding before the agent runs produce fewer reruns? Does spec quality correlate with first-run success? Does the team that invests in upfront alignment ship more predictably than the team that skips it?

The platform creates the visibility to answer those questions. The argument the book has been making from the first chapter, shift left and invest in understanding before a line is written, gets tested continuously instead of once a quarter by hand. There's no need to promise a specific result. The data just needs to exist.

A team that's an outlier in rerun rates gets a signal they didn't have before. It's a question, not a blame mechanism, and it leads somewhere actionable. A dedicated session on prompting quality. A closer look at whether the Spec Session is producing shared understanding or just producing a document. A review of spec size: are tickets too large for the agent to act on reliably, or too small to carry enough context?

The platform team can facilitate that. They know which teams improved their rerun rates and what changed, and they can bring that to the outlier team, not as a top-down intervention but as "here's what we've seen work elsewhere."

The Alexandria Problem in reverse: knowledge flowing from the center outward, continuously, as the organization learns what good looks like.

The line is thin, and Goodhart's Law, from the Oracle chapter, applies: the moment the rerun count becomes a comparison, a leaderboard, a target a manager asks a team to improve, specs get narrow, ambitions shrink, and the number gets better while the learning stops. The signal works because the team reads it. The platform only carries the mirror.

---

Charity Majors reached the same conclusion from the other end of the loop.[45] Code, she argues, is a materialized view of understanding: useful while it's current, disposable once it goes stale, and never the actual product. What teams have always been producing is the shared understanding and the behavior of the system in production. Organizations used to be limited by how fast they could ship; now they're limited by how fast they can validate and understand what they shipped. Her discipline lands on telemetry and evals, the right-hand side of the loop where what shipped meets reality. This book's lands on the spec, on the left, in the room. Different placement, same scarce thing. A spec nobody understood doesn't get safer because

[45] Charity Majors, "AI Demands More Engineering Discipline. Not Less." (charitydotwtf.substack.com, June 2026).

the telemetry after it is precise, and precise telemetry doesn't help if nobody held the intent behind what shipped.

That convergence is worth something, and it isn't validation. Nobody asked her, she wasn't evaluating my claim, and by this book's own standard, agreement that costs nothing to give doesn't weigh much. What makes it informative is the direction. The hype cycle pushes one way, toward more autonomy, fewer humans, the agent running unsupervised, and she went the other. An unfashionable conclusion reached independently is worth more than a fashionable one reached in company. Though converging voices are the ones you hear about, and the people who reached the opposite conclusion didn't write to me.

---

A platform can make the cost visible. It can't make the choice. The data can show that the room is emptying, but whether it gets filled again is the one decision no instrument makes. That's the astrolabe: it tells you where you are, but never where you go.

# The Phoenix

*The phoenix is a mythical bird that burns at the end of its life and rises from its own ashes. Not despite the destruction, but because of it.*

*If all of this is possible, what do you choose?*

If you've felt something was wrong but couldn't point to it, this book was written for you.

Not for the confident adopters. Not for the organizations announcing their AI transformation. For the engineer who is moving faster than ever and somehow feels further behind. The team lead whose team is still there on the org chart but the room has gone quiet. The person who senses the ground shifting and doesn't know yet whether to call it progress or loss.

---

We've been here before.

At some point in your team, your organization, or your career, someone decided that process and structure were slowing things down. The planning meetings, the reviews, the deliberate conversations before anyone wrote a line of code. Overhead. Friction. Things that got in the way of shipping.

So they went. Or they shrank. Or they became ceremonies that everyone attended without really being present. And for a while it felt like freedom. Faster. Lighter. More autonomous.

The decisions that used to get made in the room started getting made alone. Each one made sense in isolation. Together they didn't fit.

XP didn't lose an argument. It lost a cost calculation. The practices demanded a colocation many organizations couldn't provide, scaled badly past the single team, fit distributed work poorly, and the whole bundle was priced in the scarcest resource of that era: human implementation time. Cutting the conversation bought real capacity. Teams dropped the practices for reasons, not out of forgetfulness.

---

AI is offering the same deal at a larger scale.

Move faster. Skip the room. Optimize the individual. The tool is right there, frictionless, never tired, always ready to help. Why wait for the Spec Session? Why slow down for validation? Why sit in a room with people when the agent is ready to run?

The illusion is more powerful now. The output is real. The review requests are real. The tokens are burning, the tickets are closing, the demo channel is full. It looks like progress from every angle that gets measured.

But the room is emptier. The understanding is thinner. The junior is learning alone. The senior is prompting alone. The architect is deciding alone. And somewhere in the gap between all that individual speed, the system is accumulating something that will surface later as something nobody can quite explain. You can feel it before you can name it, and that feeling is the signal.

---

What changed is the denominator. XP's practices were priced against implementation time, and that is no longer what the room is priced against. Whether your team has collected on that is the question Appendix A prices. The rediscovery isn't nostalgia. The economics are flipping, unevenly, and the unevenness is the whole problem.

This book is an argument for a different choice.

The argument isn't against AI, and it isn't against speed. It's against the assumption that the team is overhead. Against the idea that shared understanding is a luxury you can't afford when the agent is ready to run. Against the optimization that removes the friction without noticing that the friction was doing something.

The Spec Session is slower than prompting alone. The Agora is harder to sustain than a solo workflow. The room where people say what they actually think is more fragile than a process that runs without it.

Teams matter, not as an abstract principle, but because the forest produces something the desert cannot. Because the junior watching the senior think out loud is building something that no prompt will ever teach. Because the pushback that changes your direction before the agent runs is cheaper than the rework after. Because the room where uncertainty is discussable is the room where the right thing gets built.

And this isn't only conviction. The numbers from the Oracle chapter point the same way[46]: DORA's later data shows the throughput gain is real, but delivery instability keeps climbing, and a controlled trial found experienced developers working slower with AI even as they felt faster. Speed and safety turned out to be two different metrics, and only one of them moved. The forest is still a bet, and predictability is how a team finds out whether it's paying.

---

The question is whether we choose before the room goes completely quiet, or after.

---

If you feel lost, you are not behind. You are noticing something that the metrics aren't capturing and the dashboards aren't showing.

The Phoenix doesn't rise by accident; it chooses to. That choice is the difference between what happened to Sisyphus and what can happen here. Sisyphus had no choice but to push the boulder. Teams do. Organizations do. Individual engineers do. The drift toward the desert, toward individual speed, toward the ever-agreeing genie: none of it is inevitable. It's a series of small choices that compound quietly until the room goes empty.

[46] DORA's 2024 and 2025 reports (*Impact of Generative AI in Software Development*; *State of AI-assisted Software Development*), and METR's July 2025 controlled trial. All three are discussed, with citations, in the Oracle chapter.

The renewal is also a choice. Keep the team. Protect the room. Slow down for the conversation that feels like it's getting in the way, because that conversation is often the work. Push back when the metric replaces the judgment. Say the thing that's uncomfortable to say in the Spec Session. Ask the question that surfaces the XY problem before the agent runs.

Choose the forest. Even when the desert feels faster.
Especially when it feels faster.

Understanding what to build, together, before anyone builds it was always the hard part. AI didn't change that; it just made it easier to forget.

We can choose to remember.

# Appendix A: What the room costs

Every argument in this book costs somebody a morning, and sooner or later the person who approves calendars is going to ask what that morning buys. This is the arithmetic I use when the question comes. It is not a business case, and the numbers in it are placeholders for numbers that belong to the team doing the counting.

Cost first, because that is the part nobody disputes. Five people in a room for seventy-five minutes is a little over six hours of engineering time. Add the preparation somebody does beforehand and the carry the Navigator does around it, and one gated spec runs to about seven engineer-hours. Two in an ordinary week is fourteen hours out of a five-person week of roughly two hundred, which is seven percent of the team's capacity. That looks bad written down and it should.

It is also the wrong number, because it prices the room as an addition and almost none of it is. The session inherits planning's content and can inherit its hours. What it does not inherit is planning's arithmetic, which has nothing left to do once capacity stops being the constraint. It absorbs refinement, whose one surviving question the gate answers. It absorbs the ticket-writing that used to happen afterwards, and some part of the review that was catching spec failures a week too late. What those already cost is what the seven percent has to be measured against. A team spending four hours a week on planning and refinement is paying three hours net, not fourteen, and the gross figure was mostly a relabeling.

The subtraction only happens if the old meetings actually stop. A team that adds sessions on top of planning and refinement pays gross, and gross is what seven percent looks like. That one is easy to check: if the calendar did not get emptier somewhere, the swap did not happen.

A second thing softens it further: where the agent has actually freed capacity, those hours come out of the dividend, not the principal. Whether it has is your team's number, not mine. Where it hasn't, the room is spending principal, and the seven percent is exactly what it looks like. Anyone approving the calendar does that multiplication before they read anything else, and a figure I decline to print is a figure they will assume I am hiding.

The other side of the ledger gets priced with my own wreckage rather than with a statistic I would have to caveat into uselessness. The split-PR round trip is the ordinary case, and it is the one the ratio below runs on: days rather than weeks, two people, call it twenty engineer-hours, and it also cost goodwill that never appeared on a ticket anywhere. JoustMania's player-history feature is the harder case, and I have to price it twice. Storage, pipelines, recording, replay, all of it built and none of it merged, because what got built was not what anybody had wanted. The building was the cheap half. An agent wrote most of it across a handful of evenings, and if this book is right about implementation, that number only keeps falling. What did not fall is the rest: several weeks of calendar time, a review request that sat open because nobody could close it, the attention of everyone who looked at it and could not say why it felt wrong, and a token bill nobody counted. The forty engineer-hours I would have quoted a year ago are the wrong figure now, wrong in the direction that costs me the argument, so they are out of the ratio below rather than dressed up as still comparable.

Set that against seven hours a session and the room pays for itself when it catches one twenty-hour misbuild in three specs. That is the flattering version, and it is not the honest one. The question is not how often the team builds the wrong thing. It is how often the team builds the wrong thing in a way that a room full of people could have seen coming. Some answers only appear by building, which the Agora concedes, and this appendix has to concede it too. So discount the frequency by whatever fraction of the team's misbuilds were foreseeable at spec time. If half of them were, the bar moves to one in one and a half.

Which leaves a conclusion I would rather not have reached. Ordinary rework does not carry the room on its own, because an ordinary misbuild and the room itself get priced in the same currency, and a twenty-hour swing either way was never going to decide anything. What carries it is the expensive wrong turn, the JoustMania kind, the one that runs for weeks before anybody says it out loud and never reduces to a clean number of hours at all. Implementation got cheap. Discovering it was wrong didn't, and that gap is the column below that the arithmetic cannot count. How often those happen is not a number I can supply, and anybody who has shipped for a few years has a private answer that is worse than the one said in the retro.

Break-even is a threshold and not a target. A team that starts counting caught misbuilds in order to justify its sessions will find them, and the room has become a demo channel with a spreadsheet attached.

The uncounted column runs both ways. Against the room: calendar fragmentation is real, three sessions in a day is a ceiling rather than a target and wrecks people when a team treats it as one, and a session can produce a document nobody understood while it was written and nobody opens afterwards. For the room: the junior who learns the shape of the system by watching the argument, the onboarding that runs in weeks instead of months, the departure that does not take the system out of the door with it. Assigning those a number to win the column I want to win is the move the Oracle warns against, so they stay uncounted and this stays incomplete.

Testing it needs no platform. For one quarter, count gated specs so you know how much of the practice you ran, then count agent reruns, tickets that come back as rework, and how often the team hit the date it named. Compare against the quarter before, which was badly recorded and contaminated by everything else that changed. A team that starts counting rework also starts finding it, so the old baseline is understated and any improvement will read smaller than it was. That error runs against the argument here, which is the direction I would rather have it run.

That is the only error I had been naming, which is a convenient one to have picked, because the errors running the other way are larger. A team that knows it is being measured improves while the measuring lasts, and this measurement arrives with a name attached and a book behind it. Teams reach for something like this when the pain is already bad, and bad quarters revert on their own. Nobody adopts sessions and nothing else, so the quarter will also contain a new lead, a departure, a reorganization, a better model, and that last one moves reruns down while the room takes the credit.

Which means the falsifier needs a clause it did not have. If I am allowed to decide afterwards that the sessions were not really sessions, nothing counts against me, and the paragraph in the first chapter was for show. So the terms are set here instead, before anybody has seen a number: the gate was applied, tickets went back when it failed, and the gated-spec count is the one the team set out to run. On those terms a flat quarter is a flat quarter, and the fault is mine rather than the team's.

Which leaves the number that matters most, the one that would show all of this to be wrong. A quarter of full sessions and not one of those three moves. If that happens, the arithmetic above was decoration. I would rather hand over the figure that breaks the argument than the one that sells it.

# Appendix B: The Spec Session, a working template

*A starting shape, frozen at printing. Take it, run it, and rebuild it for your team.*

I don't have years of experience running these. Nobody does, because the ritual is younger than the problem it solves. What follows is a construction from the rituals we know, and what it inherits from is sprint planning. Planning did two jobs: work out how much fits, and work out what the work actually is. Capacity stops being the constraint when the agent implements, so the first has nothing left to do. The second is the whole session, run with mob programming's mechanics. Run it, break it, tell me where it fails.

## Cadence and size

60 to 90 minutes, hard stop. Overrun means the ticket wasn't ready or is too big, and both are findings, not failures.

This is not a weekly ceremony. A team might run two or three of these in a day, with real breaks in between, because the session is the work now, not the meeting before the work. Mob programming already taught this lesson: sustained group thinking exhausts faster than solo implementation, and a tired room produces a worse spec.

Some days most of the day is spec sessions, and that's the team doing engineering.

Three to seven people. Below three, someone is prompting with witnesses. That rule is about the standing session, not the first try: the pair experiment the Agora's door opens with is deliberately below this floor, and it's how you find out whether your team wants the real thing. Above seven, the session splits by domain, and the team accepts the coverage tension that comes with splitting.

One session, one spec. No batch refinement of a backlog.

## The fast path

Need the fast path? Skip the template. Grab one coworker, agree on intent and non-goals on a single scratchpad, and run the agent. Use the full template when the work crosses boundaries or involves more of the team.

## What this replaces

The session is not added to the ceremony stack; it retires most of it. Planning is settled above: half inherited outright, half left with nothing to do. Refinement goes with it, because the only sizing question left is whether the work fits in one session, and the gate answers that. That leaves the daily. Audit it: much of what a standup catches is fragments of understanding traded in status form, and the session now does that work properly, per piece of work. Keep the daily if it still earns its fifteen minutes; on many teams it thins to a blocker-ping. Then count ceremony hours before and after. The seven percent in Appendix A is not always net-new spend; for a team running full Scrum it is often a net cut. If the calendar grows when you adopt this, you didn't adopt it; you added a meeting.

## Roles, rotating per session

*The session lead* owns the room that day, not the product forever. They close discussions, call the timebox, and decide when a disagreement gets logged instead of resolved.

*The driver* types the spec on the shared screen and types only what the group says. The driver's confusion is a feature, because a sentence the driver can't write down is a sentence the agent can't run with. This is strong-style pairing's law applied to planning: for an idea to reach the document it must pass through someone else's hands. The credit belongs to the mob lineage of Woody Zuill and Llewellyn Falco.

If you use AI during specing, you use it together. One model, on the shared screen, driven by the driver. It can challenge the spec, surface prior art, play the agent reading the draft. What it can't be is a private second opinion, because the moment people validate against their own tab instead of the room, the session is just parallel prompting with attendance. This AI is not the agent that runs after the gate, it's a tool the group wields while the spec is still soft.

This rule exists because of what a single private prompt did to a mob session. The Alexandria Problem carries the scene.

*Everyone else navigates.* Intent, edge cases, system constraints.

The Stakeholder Navigator is deliberately absent unless intent needs clarifying. Their job is protecting the room, and their presence inside it turns the session back into a requirements handoff.

## Structure

*Intent, 10 minutes.* One person states the why in plain language, uninterrupted. The group's only job is to find where they disagree with it. No solution talk yet. The friction is scheduled here, not hoped for.

*The walls, 15 to 20 minutes.* Constraints, non-goals, blast radius. What must not change, which systems this touches, what the agent must not decide on its own. This is where system thinking transfers, because it's where the invisible constraints get named out loud.

*The gaps, 20 to 30 minutes.* The core, and where the behavior gets written. The group asks the questions the agent won't ask. Edge cases, failure modes, what happens when. Every answered question becomes a sentence of behavior in the spec. Every unanswerable one becomes an explicit open decision with an owner. By the end of this phase the what exists, built question by question instead of drafted upfront.

*Read-back, 10 minutes.* The driver reads the spec aloud, top to bottom. Then the lead asks each person, by name: consent or objection. Silence isn't an answer, everyone answers. Objections get logged or resolved, and consent gets recorded.

*The gate, 5 minutes.* Intent stated. Non-goals listed. Edge cases answered or explicitly deferred. No sentence a new team member would misread. If the gate fails, the ticket goes back and the agent doesn't run.

## Output

One spec document, versioned, in the repo next to the code. The sections mirror the session: Intent, Constraints and Non-Goals, Behavior and Edge Cases, Open Decisions. Open Decisions is the honest section, because it tells the agent and the reviewers where judgment was deferred on purpose rather than omitted by accident.

## Between the gate and the run

The spec is visible before the agent runs, and the Stakeholder Navigator carries it to whoever the outcome touches. This is the review, moved. Stakeholders used to inspect the increment after the build, because the build was the expensive part, and now the spec is the increment worth inspecting. An objection now costs a conversation, an objection after the run costs the run, the review of the run, and a little trust. So the window is short, the default is the agent runs, and an objection is a new session, not a veto scribbled in a comment. Nobody approves the spec, because approval is a handoff wearing a safety vest.

Distributed teams: the async adaptation of this template is Appendix D.

## What the template doesn't solve

The run will surface things the room didn't. That's another session, not an edit, and it's why sessions have to stay cheap enough to re-run.

There is no estimation ritual anywhere, no points, no poker. The sizing question moved, it's no longer how long this will take, it's whether this is small enough to spec in one session. That sizing is the estimate, made together at the gate. Commitment moved with it. The team no longer commits to estimated capacity, it commits to specs through the gate. The when comes from flow, how many gated specs the team finishes, counted instead of guessed. The count is how the forecast gets made. What gets measured is whether it held. And the count only means something because the session cap keeps the units roughly comparable, the gate normalizes what it passes. The #NoEstimates crowd got here first, Woody Zuill again, with Vasco Duarte writing the book: make the items small and similar, and the counting does the forecasting.

A room can nod along the way a model does. The consent round forces everyone to answer, but consent can be performed just like silence can. The rotating lead helps, and it doesn't fix it.

# Appendix C: A session that failed the gate

Appendix B describes the ritual. This is one running, on a ticket from this book: JoustMania's player history, the wrong turn Appendix A could not put a clean number on, built in full and never merged.

Two warnings before it starts.

This is a reconstruction, not a transcript. The session never happened, which is exactly why the feature got built. I am writing the room I did not have, about a failure I already know the ending of, which is the most flattering possible position to write from. The test is not whether this room would have caught it. Of course it does; I built it to. The test is whether the questions it asks are the ones a room asks, and whether you have sat in a session where nobody asked them.

The second warning is that I have put three people in a room that had two. Me and a university friend, different schedules, both of us wanting the feature. The book's own floor is three, and the third seat here is the one that asks the question that ends the session. That is convenient. Take it with the appropriate suspicion, and notice that the convenience is the claim: a pair has no one who can see its framing from outside, and this is what that costs.

---

## The ticket as it arrived

*Remember how players have played before, and adapt the game over time.*

That is the whole ticket. It is a good idea, it is one sentence, and every person who read it understood something slightly different by it.

## Intent (10 minutes)

One person states the why, uninterrupted.

Returning players should find the game has learned something about them. Someone who plays aggressively gets a game tuned for that. Someone who has played ten times should not get the same experience as someone picking up a controller for the first time. The game gets to be a little different for each person over time.

The group's only job is to find where they disagree with it.

The first disagreement arrives on the word *returning*. Returning to what? To the same evening, the same party, the same living room six months later? Three people in the room, three different pictures, all of them reasonable, none of them stated in the ticket. Nobody has proposed a solution yet and the intent is already three intents. This is the friction being scheduled rather than hoped for, and it costs a few minutes of the ten.

## The walls (15 to 20 minutes)

Constraints, non-goals, blast radius.

**Must not change:** the joust detection loop. Motion thresholds, the accelerometer read, the thing that decides you moved too fast and you are out. It works, it is the game, and it is not what this ticket is about.

**Non-goals:** accounts, logins, profiles, leaderboards. Nothing that adds a step between picking up a controller and playing.

**Blast radius:** session lifecycle, whatever holds state between rounds, storage that has to survive a power cycle.

And then the constraint nobody wrote in the ticket, surfacing because the walls phase exists to surface it: a party game has no registration step. Not "we would prefer not to add one." There is no moment in the physical situation, six people, a living room, controllers coming out of a bag, where anyone would stop to identify themselves. Adding that step does not add friction to the game. It changes what the game is.

That constraint is now written down, out loud, in front of everyone. Hold onto it.

## The gaps (20 to 30 minutes)

The core, where the behavior gets written. The group asks the questions the agent will not.

*What happens when a player hands their controller to someone else mid-session?*

They do. Constantly. That is most of what happens at a party.

*What happens when the same controller gets picked up by a different person the next time the game runs?*

Also constantly. The controllers live in a bag.

*So how does the system know a player has returned?*

The room works at this one for a while, because it feels like it must have an answer. Pairing is per-controller and persists. Motion signature? People play differently when drunk, tired, or on a rug. Ask at the start? See the wall we just built.

The driver, who types only what the group says, tries to write the behavior sentence:

When a player begins a round, the game loads that player's history and adjusts difficulty accordingly.

And stops. Because *that player* has no referent. The system has a controller ID. It does not have a person, it has never had a person, and there is no cheap path from one to the other that survives the constraint on the wall.

A sentence the driver cannot write down is a sentence the agent cannot run with. That is the whole rule, and this is what it looks like when it fires.

What the room can write, once it stops trying to write the other thing:

The system can attach state to a controller across rounds within a single power-on session.

It cannot attach state to a person, in any session, ever, without a registration step this spec has ruled out.

Both true. Both writeable. Together they say the feature as scoped has no anchor.

## Read-back (10 minutes)

The driver reads the spec aloud, top to bottom. It takes less time than usual, because there is less of it than anyone expected an hour ago.

The lead asks each person by name: consent or objection.

Two consents. One objection, from the person who asked what *returning* meant: the intent as stated cannot be built under the constraints as stated, and everything downstream of that is us agreeing to build something we have not described.

Logged, not resolved.

## The gate (5 minutes)

**Intent stated.** Yes, and now visibly three intents wearing one sentence.

**Non-goals listed.** Yes.

**Edge cases answered or explicitly deferred.** *No.* The controller-handoff case is not an edge. It is the ordinary case, it has no answer, and deferring it defers the feature.

**No sentence a new team member would misread.** Not applicable to a spec that does not describe a buildable thing.

The gate fails. The ticket goes back. The agent does not run.

One session, inside the timebox. Nothing built. Several weeks of calendar time not spent.

## Open decisions

The honest section, which is the point of having it.

- **Is there a version of this that attaches to the controller rather than the person?** Owner: me. A controller that remembers its own last three rounds is a different feature, possibly a smaller and better one, and nobody in the room could say whether it is interesting.
- **What was the actual need?** Owner: the person who raised the objection. The ticket says *player history*. Nobody in the room could state the problem that solves.
- **Does the demo need adaptation at all, or does it need to look adaptive on stage?** Owner: me. Different question, different build, and it was never asked.

## What went back

The ticket came back a week later, reframed by the second open decision.

The need underneath was not player history. It was that the game feels identical on the tenth round, and a demo where nothing changes is a demo about nothing. Both of those are about *the session*, not about *the person*. State the need that way and the identity problem disappears, because there is nothing left that needs to know who is holding anything.

That version is smaller, it is buildable under every constraint on the wall, closer to what the talk was actually about, and it is the version a room reaches in an hour. Two people, heads down and enthusiastic, did not reach it in several weeks.

The gate failing is not the session going wrong. It is the session working. The ticket that goes back is cheaper than every ticket that goes through and should not have.

---

*One more time, because the arithmetic is the point: one session against the weeks Appendix A prices. The agent wrote it in an afternoon. That was never the expensive part.*

# Appendix D: Async Spec Planning

The Agora names the form and its sharpest risk. This is the full treatment, for the teams the Curse of Sisyphus was about, the ones the problem damages most: distributed, different timezones, permanently async. They deserve more than a synchronous prescription.

The session becomes a review request on the spec. The draft opens. Comments come in. The what, the why, and the not-building get challenged asynchronously. The merge condition is when the team has converged, meaning the concerns have been addressed and the understanding is shared.

The phases map directly. The draft carries intent, constraints, and non-goals before it opens. The comment thread is the challenge phase, and silence is still not an answer; every named reviewer responds or the spec doesn't merge. The consent round is the approval set, with objections as blocking comments. The timebox becomes a deadline: two working days, not two weeks.

The room's mechanics have to be replaced, and one of them can't be. In the room, an idea reaches the document only by passing through someone else's hands: the driver types what the group says, so no framing reaches the document unfiltered. Async has no filter. One person holds the pen, their framing is the document, and everyone else reacts to it, which is the handed-spec problem the Agora warns about, built into the form's first move. The thread's challenge compensates only partly; that is the honest cost of working apart.

What can be replaced: the read-back becomes the revision note, where the owner writes what changed under challenge and why before merging. A spec that merges unchanged is the async version of a room that nodded. And the lead's clock becomes the merge owner, rotating like the room's lead: someone whose job this round is to call convergence at the deadline, not to win the thread.

The mitigation for the risk the Agora names is minimum viable context, applied to the spec proposal itself. A proposal rich enough that a reviewer cannot respond meaningfully without actually thinking about it. One that surfaces the assumptions, names the edge cases, explains the reasoning behind the decisions, demanding engagement rather than permitting rubber-stamping. A spec drafted by an agent and reviewed by an agent merges without anyone having understood it; the review request is the session, and if no human thinks in it, it didn't happen.

The same culture requirement applies. The async version needs people to actually comment, actually raise the concern, actually say "I don't think this covers the edge case where X," rather than approving because nobody wanted to slow things down or because an AI generated a plausible-sounding approval. A blocking comment can be performed just like a consent round can; the form guarantees a response, not a thought.

The test for whether it worked is not whether the review request merged; it is the same test the room uses: whether the team could explain the decisions without the document. The fuller async build, as it evolves, lives at leftoftheloop.dev.

# Appendix E: Further reading

The landscape moves faster than a book can, so this is a snapshot rather than a survey.

*Tools:* GitHub's Spec-Kit (github.com/github/spec-kit), the BMAD-Method (github.com/bmad-code-org/BMAD-METHOD), and Matt Pocock's AI Hero (aihero.dev), whose `/grill-with-docs` skill makes the Spec Session executable.

*Concept:* "Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants" (arxiv.org/abs/2602.00180, 2026).

*Process:* Simon Martinelli's AI Unified Process (unifiedprocess.ai), set out in *Spec-Driven Development: From Specs to Code with AI Agents* (Apress, 2026). It is the most developed account of what happens after the room, and its loop starts one step to the right of this one: the workshop that produces the shared picture is named as the thing tools cannot replace, then sits outside the life cycle it feeds. Where the two part is what alignment runs on afterwards. Martinelli puts it in the artifact; this book leaves it with the people who built it.

*Risks:* ThoughtWorks' Technology Radar, Volume 33 (2025) flags that the workflows "remain elaborate and opinionated" and that some tools "generate lengthy spec files that are hard to review."

*Closest external framing:* Kief Morris, "Humans and Agents in Software Engineering Loops," "Exploring Gen AI" (martinfowler.com), on humans "on the loop," building and maintaining the harness.

Morris solves where humans sit in the process; this book solves what they need to do before it starts.

A running version of this list lives at leftoftheloop.dev.

# Acknowledgments

I hope this book was worth your time.

I started writing because I thought I had ideas about AI. I finished realizing the book was really about teams. It was, along the way, a kind of self-reflection: an accounting of where I took the easy route, and where I should have been more dedicated to the team than to myself.

A lot of books in this genre can sound like utopian dreams. Extreme Programming showed us what collaboration could look like, and much of it was applied, just never all of it, and rarely in the same way twice. This book will meet the same fate. That is fine. If you found something useful here, or even just enjoyed the anecdotes, then it did what it needed to do.

This book stands on older shoulders. Kent Beck and the XP community made the diagnosis long before agents existed. Woody Zuill and Llewellyn Falco showed what a whole team thinking together looks like in practice. I have only moved their furniture into a new house.

My thanks also go to the first readers, from the earliest drafts to the more refined ones. Matthias asked "why do I need the team?" and refused to accept my first three answers. The book is sharper because he kept refusing. Kaushal gave me the insight that implementation was never just a bottleneck, but the medium through which knowledge traveled; half of the Alexandria Problem grew from that sentence. Johann read for the readers I would otherwise have forgotten. And to everyone else who gave me their evenings and their honest comments: thank you. Where the book is right, they helped. Where it is wrong, that is mine alone.

Thank you to my family, who lost me to this manuscript on more weekends than I want to count.

AI agents helped me edit, structure, and stress-test these pages. The mistakes, like the ideas, are mine. I tried to stay left of my own loop.

One more thing, because the pages ahead will age: the tools in this book will be renamed or replaced before its ideas are. Teams outlast tooling. That is the bet underneath every chapter, and the safest one in the book.

Now take the pieces that matter to you and try them. Share what you find, tell me where I am wrong at leftoftheloop.dev, and do not let the tool define how you work.

# Glossary

The terms this book leans on, defined once. Some have ancestors, credited in the chapters that introduce them. The definitions here are how this book uses them.

**The Agora** — The condition that makes a room more than a collection of individuals: the shared awareness a team builds before acting, and the disposition to keep building it. Not the session itself; what the session exists to produce. Named in the chapter of the same name.

**The Alexandria Problem** — Not an event but a process: the library doesn't burn in one dramatic fire, it burns one skipped conversation at a time. The knowledge that used to accumulate through proximity and repetition stops the moment AI removes the friction of asking. Named for what happens when nobody notices the smoke until it's already gone.

**Constructive friction** — Disagreement from someone independent of your framing, arriving before the agent runs. The mechanism that tests shared understanding, and the thing an ever-agreeing tool cannot supply. Not an obstacle to remove; the reason the team's judgment is worth more than the model's agreement. Its counterpart is *weighted validation*, the same independence spent on agreement instead of dissent.

**Direction** — The running answer to "where does this need to go," held closely enough that the next decision can be checked against it: whether today's choice still makes sense six months from now, whether the system is becoming one thing or three. Judgment, once it's shared.

**Epistemic safety** — The conditions where people reveal what they don't know. Sharper than psychological safety: the junior asks without calculating the risk first, the senior shares the reasoning and not just the conclusion, the half-formed thought gets voiced. What the room runs on.

**Extreme Programming (XP)** — Kent Beck's practice from the nineties, built on the conviction that software development is a social activity: pair programming, collective ownership, the planning game. This book leans on it as diagnosis rather than prescription. XP named what the collaboration was for long before agents existed, and lost on a cost calculation rather than an argument. What returns here returns repriced.

**The forest and the desert** — Two working environments, borrowed from Beth Andres-Beck and Kent Beck. The desert is open, fast, unobstructed, where you can move without friction; the forest is dense and slower, where the ecosystem is richer and learning accumulates. AI is the most powerful desert tool ever built, which makes the forest more important than it's ever been, and more fragile. The forest earns its place by surviving what the desert can't.

**The gate** — The pass condition that ends a Spec Session: intent stated, non-goals listed, edge cases answered or explicitly deferred, no sentence a new team member would misread. Also the unit of commitment: the team commits to specs through the gate, not to estimated capacity.

**Left of the loop** — The loop itself is the agent's: implementation, review, iteration, at machine speed. To its right sits production, where the output meets reality. To its left is where the thinking happens: what to build, why, and what done looks like. Left of the loop is the human work, the shared understanding built while the work is still on the page. This book is an argument for that side.

**Minimum viable context** — As little as possible, as much as needed. The right context, not just enough of it: what's assumed, what's load-bearing, what would change the answer if it were different. Includes surfacing the actual problem before asking about the assumed solution.

**Predictability** — Whether the team can tell a stakeholder when something will be done, and then do it. Not velocity, not throughput, not raw output: the metric that can't be gamed by one person and an agent for an afternoon. A team metric, not an individual one, because it takes shared understanding to earn.

**Product thinking** — Challenging intent, representing user needs, asking why before the team builds what. Half of what the old Product Owner carried, and the half that distributes: when the team builds the spec together, it becomes a team competency rather than one role's job. The other half, the organizational interface, stays with the Stakeholder Navigator.

**Recognition work** — Catching the wrong shape before it compounds: the drift from intent nobody had encoded yet, before there's a test to write. Quiet, constant, and learned from being in the system: the check an eval can't run.

**The room** — Wherever a team builds shared understanding together. A condition, not a location; the best room can have no walls at all. It has to be made, and protected.

**The spec** — Shared understanding of what to build and why, held in common. Not a document; the document is the record of the convergence. Where the formal sense is meant — a contract, like the OpenFeature specification — this book uses the full word.

**Spec review** — A review request on the spec instead of the code: the defined moment where the need gets validated before anything gets built. A checkpoint, not a planning ceremony, where the stakeholder can say "that's not what I meant" while it's still cheap to hear. The mechanism that closes the gap between what's asked and what's needed. Compare *Spec Session*, which is where the team builds the picture; the review is where whoever the outcome touches confirms it was the right one.

**Spec Session** — The deliberate moment where a team converges on a spec before the agent runs: intent challenged, constraints named, edge cases answered, consent recorded. It inherits sprint planning's content and mob programming's mechanics, and it replaces the walk to the desk. When the room can't be shared, the same convergence runs async, as a review request on the spec. Compare *spec review*, which validates the need rather than building the understanding. A working template waits in Appendix B.

**Stakeholder Navigator** — The role that owns the organizational interface after product thinking distributes into the team: carries the spec to whoever the outcome touches, manages the review cycle, and protects the room from the noise while the team thinks. The part of the old Product Owner that doesn't distribute.

**The three functions** — The three kinds of attention a Spec Session needs in the room: someone holding the system direction, someone watching the ground for the edge cases that drift when nobody is looking, and someone managing the outside so the room stays protected long enough to think. The agent is the fourth, running only after the other three have done their work. The seats overlap and one person can hold two, but the minimum viable unit is three.

**Weighted validation** — Agreement from someone independent of your framing, with their own hard-won experience and their own reasons to disagree, who could have said no. It changes something an AI's agreement can't, because it costs the person something to give. Something only another person can provide, before the agent runs. Its dissenting form is *constructive friction*.

**The XY problem** — Someone has problem X, thinks the solution is Y, and asks for help with Y without saying what X was. The answer to Y lands and solves nothing, because the person helping never found out about X. Coined by Mark Jason Dominus; minimum viable context is what prevents it.

# About the Author

Simon Schrottner is a software engineer who has spent more than a decade working on the tools other engineers build on top of. He maintains OpenFeature and speaks about it and the systems around it at conferences across Europe and beyond.

*Left of the Loop* grew out of a blog series argued in public, post by post, stress-tested by readers who pushed back before any of it became a book. The failures in it are his own. Working things out in public is still how he prefers to do it.

He lives in Austria and writes at schrottner.at.